COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

Department of Computing

Internet of Things Laboratory

(20MSS84)

NAME: SHREE KOTTES J

REG.NO: 7176 22 31 050

CLASS: M.Sc., SOFTWARE SYSTEMS

(8th Semester)

MAIL ID: 71762231050@cit.edu.in

INDEX

S. No	Date	Title	Sign
1.	26/12/2025	Examination Alarm System	
2.	26/12/2025	Traffic Control System	
3.	04/01/2026	Odometer	
4.	30/01/2026	Music Visualizer	
5.	30/01/2026	Motion Detection and Photo Resistor	
6.	06/02/2026	Gas Leakage Detection	

EX.NO: 1	EXAMINATION ALARM SYSTEM
26/12/2025	

AIM

To build an exam alarm system that will be able to beep every hour, half an hour, or with custom time interval.

APPARATUS REQUIRED

- Arduino Uno
- Breadboard
- RTC DS1307 module
- I2C LCD Display (16x2)
- Pushbuttons (x4) (Mode, Up, Down, Set)
- Buzzer
- Jumper wires

PRE-CONDITIONS

- Hardware components are wired correctly, ensuring the RTC module and I2C LCD are securely connected to the Arduino I2C pins (A4, A5).
- Required libraries (`Wire.h`, `RTClib.h`, `LiquidCrystal_I2C.h`) are installed in the Arduino IDE.
- The RTC DS1307 module is properly configured with the current date and time.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The LCD correctly displays the current time, elapsed/remaining time, and configuration menus.
- The user can configure alarm intervals (half-hour, one-hour, custom, or random) using the pushbuttons.
- The buzzer sounds alerts automatically at the configured intervals while the exam is running.
- The exam process starts and stops gracefully upon interacting with the Set button.

CODE

```
#include <Wire.h>
#include <RTClib.h>
#include <LiquidCrystal_I2C.h>

// Initialize LCD (address 0x27, 16 columns, 2 rows)
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
RTC_DS1307 rtc;

// Pin definitions
#define BUZZER_PIN 9
#define BTN_MODE 2
#define BTN_UP 3
#define BTN_DOWN 4
#define BTN_SET 5

// Configuration variables
enum AlarmPattern { HALF_HOUR, ONE_HOUR, CUSTOM, RANDOM };
AlarmPattern alarmPattern = HALF_HOUR;

enum DisplayMode { ELAPSED, REMAINING };
DisplayMode displayMode = ELAPSED;

enum SoundType { SINGLE_BEEP, DOUBLE_BEEP, LONG_BEEP };
SoundType soundType = SINGLE_BEEP;

// Timing variables
unsigned long examStartTime = 0;
unsigned long examDuration = 3600000; // Default 1 hour in milliseconds
unsigned long lastAlarmTime = 0;
unsigned long nextAlarmTime = 0;
bool examStarted = false;
bool alarmEnabled = true;

// Custom interval (in minutes)
int customInterval = 20; // Default 20 minutes

// Random interval configuration (in minutes)
int randomMinInterval = 15; // Minimum 15 minutes
int randomMaxInterval = 60; // Maximum 60 minutes

// Menu system
enum MenuState { MAIN_SCREEN, MENU_ALARM_PATTERN, MENU_CUSTOM_INTERVAL,
                 MENU_RANDOM_MIN, MENU_RANDOM_MAX, MENU_DISPLAY_MODE,
                 MENU_SOUND_TYPE, MENU_EXAM_DURATION, MENU_START_EXAM };
MenuState menuState = MAIN_SCREEN;
int menuSelection = 0;

// Button debouncing
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 200;

void setup() {
  Serial.begin(9600);
```

```
  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.clear();

  // Initialize RTC
  if (!rtc.begin()) {
    lcd.setCursor(0, 0);
    lcd.print("RTC Not Found!");
    while (1);
  }

  if (!rtc.isrunning()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }

  // Initialize pins
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(BTN_MODE, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
  pinMode(BTN_SET, INPUT_PULLUP);

  digitalWrite(BUZZER_PIN, LOW);

  lcd.setCursor(0, 0);
  lcd.print("Exam Alarm Sys");
  lcd.setCursor(0, 1);
  lcd.print("Ready to Config");
  delay(2000);
}

void loop() {
  handleButtons();

  if (menuState == MAIN_SCREEN) {
    if (examStarted) {
      displayExamStatus();
      checkAlarm();
    } else {
      displayMainScreen();
    }
  } else {
    displayMenu();
  }

  delay(100);
}
```

```
void handleButtons() {
  unsigned long currentTime = millis();

  if (currentTime - lastButtonPress < debounceDelay) {
    return;
  }

  if (digitalRead(BTN_MODE) == LOW) {
    lastButtonPress = currentTime;
    handleModeButton();
  } else if (digitalRead(BTN_UP) == LOW) {
    lastButtonPress = currentTime;
    handleUpButton();
  } else if (digitalRead(BTN_DOWN) == LOW) {
    lastButtonPress = currentTime;
    handleDownButton();
  } else if (digitalRead(BTN_SET) == LOW) {
    lastButtonPress = currentTime;
    handleSetButton();
  }
}

void handleModeButton() {
  if (examStarted) return; // Don't allow menu during exam

  if (menuState == MAIN_SCREEN) {
    menuState = MENU_ALARM_PATTERN;
    menuSelection = 0;
  } else {
    // Navigate to next menu
    int nextMenu = (int)menuState + 1;

    // Skip menus based on alarm pattern selection
    if (nextMenu == MENU_CUSTOM_INTERVAL && alarmPattern != CUSTOM) {
      nextMenu++; // Skip custom interval if not selected
    }
    if (nextMenu == MENU_RANDOM_MIN && alarmPattern != RANDOM) {
      nextMenu += 2; // Skip both random menus if not selected
    } else if (nextMenu == MENU_RANDOM_MAX && alarmPattern != RANDOM) {
      nextMenu++; // Skip random max if not selected
    }

    if (nextMenu > MENU_START_EXAM) {
      nextMenu = MENU_ALARM_PATTERN;
      menuSelection = 0;
    }
```

```
    menuState = (MenuState)nextMenu;
  }
  lcd.clear();
}

void handleUpButton() {
  if (menuState == MAIN_SCREEN) return;

  switch (menuState) {
    case MENU_ALARM_PATTERN:
      alarmPattern = (AlarmPattern)((alarmPattern + 1) % 4);
      break;

    case MENU_CUSTOM_INTERVAL:
      customInterval += 1; // Increment by 5 minutes
      if (customInterval > 180) customInterval = 180; // Max 3 hours
      break;

    case MENU_RANDOM_MIN:
      randomMinInterval += 5; // Increment by 5 minutes
      if (randomMinInterval > 120) randomMinInterval = 120; // Max 2
       ↪  hours
      if (randomMinInterval >= randomMaxInterval) {
        randomMaxInterval = randomMinInterval + 5;
      }
      break;

    case MENU_RANDOM_MAX:
      randomMaxInterval += 5; // Increment by 5 minutes
      if (randomMaxInterval > 180) randomMaxInterval = 180; // Max 3
       ↪  hours
      break;

    case MENU_DISPLAY_MODE:
      displayMode = (DisplayMode)((displayMode + 1) % 2);
      break;

    case MENU_SOUND_TYPE:
      soundType = (SoundType)((soundType + 1) % 3);
      break;

    case MENU_EXAM_DURATION:
      examDuration += 300000; // Add 5 minutes
      if (examDuration > 14400000) examDuration = 14400000; // Max 4
       ↪  hours
      break;
  }
}
```

```
void handleDownButton() {
  if (menuState == MAIN_SCREEN) return;

  switch (menuState) {
    case MENU_ALARM_PATTERN:
      alarmPattern = (AlarmPattern)((alarmPattern - 1 + 4) % 4);
      break;

    case MENU_CUSTOM_INTERVAL:
      if (customInterval > 0) {
        customInterval -= 1; // Decrement by 5 minutes
      }
      break;

    case MENU_RANDOM_MIN:
      if (randomMinInterval > 5) {
        randomMinInterval -= 5; // Decrement by 5 minutes
      }
      break;

    case MENU_RANDOM_MAX:
      randomMaxInterval -= 5; // Decrement by 5 minutes
      if (randomMaxInterval < randomMinInterval + 5) {
        randomMaxInterval = randomMinInterval + 5;
      }
      if (randomMaxInterval < 10) randomMaxInterval = 10;
      break;

    case MENU_DISPLAY_MODE:
      displayMode = (DisplayMode)((displayMode - 1 + 2) % 2);
      break;

    case MENU_SOUND_TYPE:
      soundType = (SoundType)((soundType - 1 + 3) % 3);
      break;

    case MENU_EXAM_DURATION:
      if (examDuration > 300000) {
        examDuration -= 300000; // Subtract 5 minutes
      }
      break;
  }
}

void handleSetButton() {
  if (!examStarted) {
    startExam();
```

```
    menuState = MAIN_SCREEN;
  } else if (examStarted) {
    stopExam();
  }
  lcd.clear();
}

void displayMainScreen() {
  DateTime now = rtc.now();

  lcd.setCursor(0, 0);
  lcd.print("Time: ");
  printTime(now.hour(), now.minute(), now.second());

  lcd.setCursor(0, 1);
  lcd.print("Press MODE menu");
}

void displayMenu() {
  lcd.setCursor(0, 0);

  switch (menuState) {
    case MENU_ALARM_PATTERN:
      lcd.print("Alarm Pattern: ");
      lcd.setCursor(0, 1);
      if (alarmPattern == HALF_HOUR) lcd.print("Every Half Hour");
      else if (alarmPattern == ONE_HOUR) lcd.print("Every One Hour ");
      else if (alarmPattern == CUSTOM) lcd.print("Custom Interval");
      else lcd.print("Random Interval");
      break;

    case MENU_CUSTOM_INTERVAL:
      lcd.print("Custom Interval:");
      lcd.setCursor(0, 1);
      lcd.print(customInterval);
      lcd.print(" Minutes      ");
      break;

    case MENU_RANDOM_MIN:
      lcd.print("Random Min Int:");
      lcd.setCursor(0, 1);
      lcd.print(randomMinInterval);
      lcd.print(" Minutes      ");
      break;

    case MENU_RANDOM_MAX:
      lcd.print("Random Max Int:");
      lcd.setCursor(0, 1);
```

```
      lcd.print(randomMaxInterval);
      lcd.print(" Minutes      ");
      break;

    case MENU_DISPLAY_MODE:
      lcd.print("Display Mode:  ");
      lcd.setCursor(0, 1);
      if (displayMode == ELAPSED) lcd.print("Elapsed Time   ");
      else lcd.print("Remaining Time ");
      break;

    case MENU_SOUND_TYPE:
      lcd.print("Sound Type:    ");
      lcd.setCursor(0, 1);
      if (soundType == SINGLE_BEEP) lcd.print("Single Beep    ");
      else if (soundType == DOUBLE_BEEP) lcd.print("Double Beep    ");
      else lcd.print("Long Beep      ");
      break;

    case MENU_EXAM_DURATION:
      lcd.print("Exam Duration: ");
      lcd.setCursor(0, 1);
      unsigned long hours = examDuration / 3600000;
      unsigned long mins = (examDuration % 3600000) / 60000;
      lcd.print(hours);
      lcd.print(" Hrs ");
      lcd.print(mins);
      lcd.print(" Mins  ");
      break;
  }
}

void displayExamStatus() {
  unsigned long currentTime = millis();
  unsigned long elapsed = currentTime - examStartTime;

  lcd.setCursor(0, 0);

  if (displayMode == ELAPSED) {
    lcd.print("Elapsed: ");
    printDuration(elapsed);
  } else {
    lcd.print("Remain:  ");
    if (elapsed < examDuration) {
      printDuration(examDuration - elapsed);
    } else {
      lcd.print("00:00   ");
    }
```

```
  }

  lcd.setCursor(0, 1);
  DateTime now = rtc.now();
  lcd.print("Time: ");
  printTime(now.hour(), now.minute(), now.second());

  // Check if exam is over
  if (elapsed >= examDuration) {
    examFinished();
  }
}

void printTime(int h, int m, int s) {
  if (h < 10) lcd.print("0");
  lcd.print(h);
  lcd.print(":");
  if (m < 10) lcd.print("0");
  lcd.print(m);
  lcd.print(":");
  if (s < 10) lcd.print("0");
  lcd.print(s);
}

void printDuration(unsigned long duration) {
  unsigned long hours = duration / 3600000;
  unsigned long mins = (duration % 3600000) / 60000;

  if (hours < 10) lcd.print("0");
  lcd.print(hours);
  lcd.print(":");
  if (mins < 10) lcd.print("0");
  lcd.print(mins);
  lcd.print("   ");
}

void startExam() {
  examStarted = true;
  examStartTime = millis();
  lastAlarmTime = examStartTime;

  // Set next alarm based on pattern
  if (alarmPattern == HALF_HOUR) {
    nextAlarmTime = examStartTime + 1800000; // 30 minutes
  } else if (alarmPattern == ONE_HOUR) {
    nextAlarmTime = examStartTime + 3600000; // 60 minutes
  } else if (alarmPattern == CUSTOM) {
    nextAlarmTime = examStartTime + (customInterval * 60000UL); // Custom
 ↪  minutes
```

```
  } else { // RANDOM
    unsigned long minMs = randomMinInterval * 60000UL;
    unsigned long maxMs = randomMaxInterval * 60000UL;
    nextAlarmTime = examStartTime + random(minMs, maxMs);
  }

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Exam Started!");
  lcd.setCursor(0, 1);
  lcd.print("Good Luck!");
  delay(2000);
  lcd.clear();
}

void stopExam() {
  examStarted = false;
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Exam Stopped!");
  delay(2000);
  lcd.clear();
}

void examFinished() {
  examStarted = false;
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Exam Finished!");
  lcd.setCursor(0, 1);
  lcd.print("Well Done!");
  playSound();
  delay(3000);
  lcd.clear();
}

void checkAlarm() {
  unsigned long currentTime = millis();

  if (currentTime >= nextAlarmTime && alarmEnabled) {
    triggerAlarm();

    // Set next alarm
    lastAlarmTime = currentTime;
    if (alarmPattern == HALF_HOUR) {
      nextAlarmTime = currentTime + 1800000;
    } else if (alarmPattern == ONE_HOUR) {
      nextAlarmTime = currentTime + 3600000;
```

```
    } else if (alarmPattern == CUSTOM) {
      nextAlarmTime = currentTime + (customInterval * 60000UL);
    } else { // RANDOM
      unsigned long minMs = randomMinInterval * 60000UL;
      unsigned long maxMs = randomMaxInterval * 60000UL;
      nextAlarmTime = currentTime + random(minMs, maxMs);
    }
  }
}

void triggerAlarm() {
  playSound();
}

void playSound() {
  switch (soundType) {
    case SINGLE_BEEP:
      tone(BUZZER_PIN, 1000, 200);
      break;

    case DOUBLE_BEEP:
      tone(BUZZER_PIN, 1000, 200);
      delay(800);
      tone(BUZZER_PIN, 1000, 200);
      break;

    case LONG_BEEP:
      tone(BUZZER_PIN, 1000, 1000);
      break;
  }
}
```

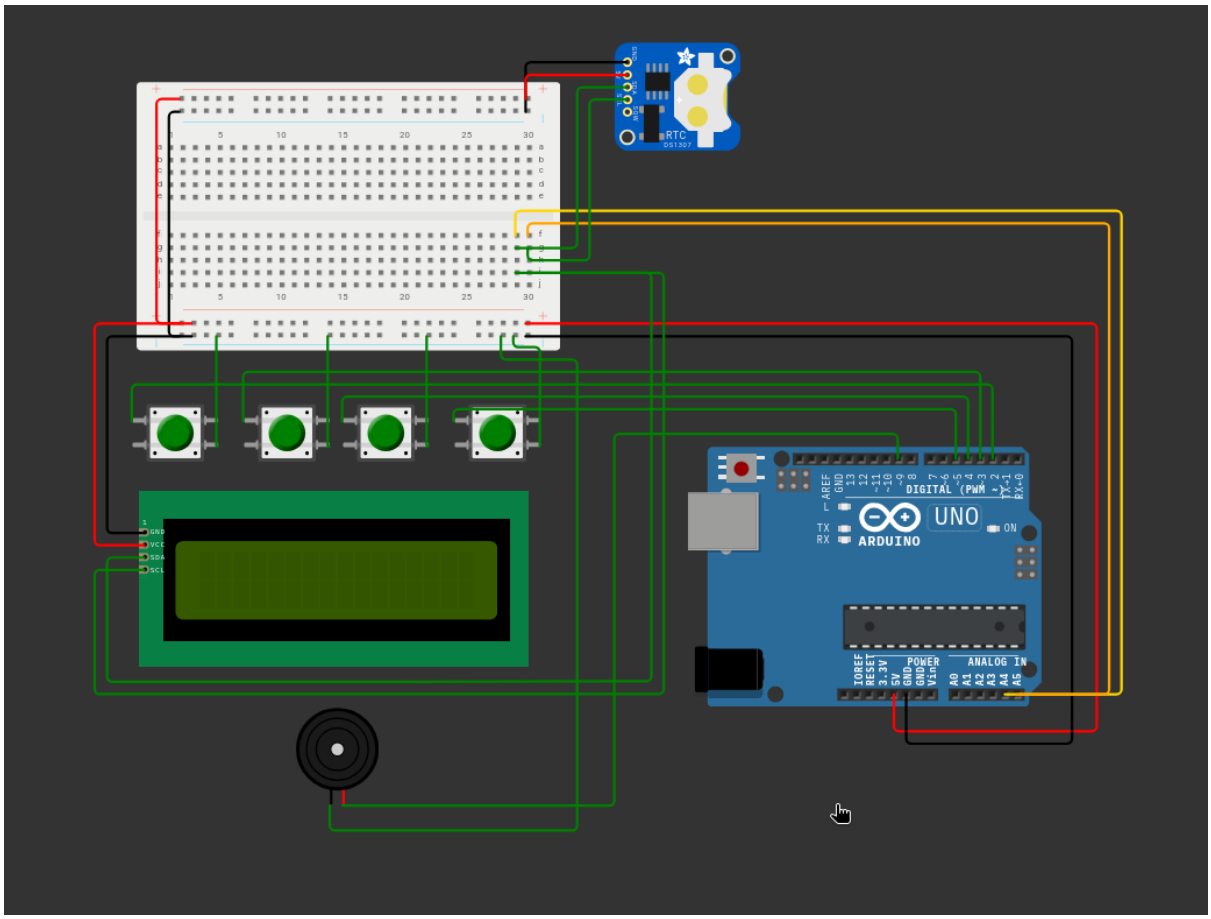
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": -204.6,
     ↪  "left": -131.6, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 77.4, "left": 297,
     ↪  "attrs": {} },
    { "type": "wokwi-ds1307", "id": "rtc1", "top": -235.8, "left": 240.1,
     ↪  "attrs": {} },
    {
      "type": "wokwi-pushbutton",
```

```
      "id": "btn1",
      "top": 44.6,
      "left": -134.4,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": 112,
      "left": -128.8,
      "attrs": { "pins": "i2c" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 44.6,
      "left": -48,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn3",
      "top": 44.6,
      "left": 28.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn4",
      "top": 44.6,
      "left": 115.2,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": 271.2,
      "left": -7.8,
      "attrs": { "volume": "0.1" }
    }
  ],
  "connections": [
    [ "uno:5V", "bb1:bp.25", "red", [ "v28.7", "h157.4", "v-315.9" ] ],
    [ "uno:GND.2", "bb1:bn.25", "black", [ "v38.3", "h128.7", "v-315.5" ]
     ↪ ],
    [ "uno:A4", "bb1:29b.f", "gold", [ "h90.6", "v-374.5", "h-470.4" ] ],
    [ "uno:A5", "bb1:30b.f", "orange", [ "h71.5", "v-364.9", "h-451.2" ]
     ↪ ],
```

```
    [ "bb1:bp.1", "bb1:tp.1", "red", [ "h-17.6", "v-174" ] ],
    [ "bb1:bn.1", "bb1:tn.1", "black", [ "h-8", "v-174" ] ],
    [ "rtc1:GND", "bb1:tn.25", "black", [ "h0" ] ],
    [ "rtc1:5V", "bb1:tp.25", "red", [ "h-67.2", "v-0.4", "h-11.2" ] ],
    [ "rtc1:SDA", "bb1:29b.g", "green", [ "h-38.4", "v124.5", "h-48" ] ],
    [ "rtc1:SCL", "bb1:30b.g", "green", [ "h-28.8", "v124.6", "h-48" ] ],
    [ "lcd1:GND", "bb1:bn.2", "black", [ "h-28.8", "v-152.3" ] ],
    [ "lcd1:VCC", "bb1:bp.2", "red", [ "h-38.4", "v-171.8" ] ],
    [
      "lcd1:SDA",
      "bb1:29b.i",
      "green",
      [ "h-28.8", "v96.2", "h230.4", "v-0.4", "h192", "v-316.4" ]
    ],
    [ "lcd1:SCL", "bb1:30b.i", "green", [ "h-38.4", "v96.3", "h441.6",
     ↪  "v-326.4" ] ],
    [ "btn1:1.l", "uno:2", "green", [ "v-28.8", "h667.9" ] ],
    [ "btn2:1.l", "uno:3", "green", [ "v-38.4", "h572" ] ],
    [ "btn3:1.l", "uno:4", "green", [ "v-19.2", "h485.7" ] ],
    [ "btn4:1.l", "uno:5", "green", [ "v-9.6", "h389.8" ] ],
    [ "bz1:2", "uno:9", "green", [ "h210.8", "v-288", "h220" ] ],
    [ "bz1:1", "bb1:bn.23", "green", [ "v19.2", "h192", "v-364.8",
     ↪  "h-59.2" ] ],
    [ "btn4:2.r", "bb1:bn.24", "green", [ "h0.2", "v-76.6", "h-20.8" ] ],
    [ "bb1:bn.4", "btn1:2.r", "green", [ "v0" ] ],
    [ "btn2:2.r", "bb1:bn.11", "green", [ "h0" ] ],
    [ "btn3:2.r", "bb1:bn.18", "green", [ "h0" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 2	TRAFFIC CONTROL SYSTEM
26/12/2025	

AIM

To build a Traffic Control System using Arduino Board.

APPARATUS REQUIRED

- Arduino Uno
- Breadboards
- RTC DS3231 module
- I2C LCD Display (16x2)
- LEDs (4x Red, 4x Yellow, 4x Green)
- 220-ohm Resistors (x12)
- Pushbuttons (x3) (Menu, Up, Down)
- Jumper wires

PRE-CONDITIONS

- The 12 LEDs (representing 4 directions: N, S, E, W) and their current-limiting resistors are wired correctly to the Arduino digital pins.
- RTC and I2C LCD are properly connected to the I2C pins.
- The required libraries (`Wire.h`, `RTClib.h`, `LiquidCrystal_I2C.h`) are installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The traffic lights correctly sequence through Green, Yellow, and Red for North/South and East/West directions.
- The LCD displays the current time, status (Normal or Peak hours), and the active traffic phase.
- The timing dynamically adjusts based on peak hours retrieved from the RTC.
- The user can adjust timings (Peak Green, Normal Green, Yellow, Peak Hours) using the menu system.

CODE

```
#include <Wire.h>
#include <RTClib.h>
#include <LiquidCrystal_I2C.h>

// Initialize RTC and LCD
RTC_DS3231 rtc;
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Address 0x27, 16x2 LCD

// Traffic Light Pin Definitions
// Direction 1 (North)
const int N_RED = 2;
const int N_YELLOW = 3;
const int N_GREEN = 4;

// Direction 2 (South)
const int S_RED = 5;
const int S_YELLOW = 6;
const int S_GREEN = 7;

// Direction 3 (East)
const int E_RED = 8;
const int E_YELLOW = 9;
const int E_GREEN = 10;

// Direction 4 (West)
const int W_RED = 11;
const int W_YELLOW = 12;
const int W_GREEN = 13;

// Button Pins
const int BTN_MENU = A0;
const int BTN_UP = A1;
const int BTN_DOWN = A2;

// Timing Configuration (in seconds)
int peakGreenTime = 20;      // Green light during peak hours
int normalGreenTime = 30;    // Green light during normal hours
int yellowTime = 3;          // Yellow light duration
int allRedTime = 2;          // All red safety time

// Peak Hours Configuration (24-hour format)
int peakStartHour = 7;       // Peak hours start at 7:00 AM
int peakEndHour = 10;        // Peak hours end at 10:00 AM

// System Variables
unsigned long previousMillis = 0;
int currentPhase = 0;        // 0=N/S Green, 1=N/S Yellow, 2=All Red,
 ↪  3=E/W Green, 4=E/W Yellow, 5=All Red
int phaseTimer = 0;
bool isPeakHour = false;
int currentGreenTime = normalGreenTime;

// Menu System
enum MenuState {
```

```
  MENU_MAIN,
  MENU_PEAK_GREEN,
  MENU_NORMAL_GREEN,
  MENU_YELLOW,
  MENU_PEAK_START,
  MENU_PEAK_END
};

MenuState currentMenu = MENU_MAIN;
bool inConfigMode = false;
unsigned long lastButtonPress = 0;
const unsigned long buttonDebounce = 200;

void setup() {
  Serial.begin(9600);

  // Initialize RTC
  if (!rtc.begin()) {
    Serial.println("RTC not found!");
    while (1);
  }

  // If RTC lost power, set to compile time
  if (rtc.lostPower()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }

  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.print("Traffic System");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);

  // Initialize Traffic Light Pins
  pinMode(N_RED, OUTPUT);
  pinMode(N_YELLOW, OUTPUT);
  pinMode(N_GREEN, OUTPUT);
  pinMode(S_RED, OUTPUT);
  pinMode(S_YELLOW, OUTPUT);
  pinMode(S_GREEN, OUTPUT);
  pinMode(E_RED, OUTPUT);
  pinMode(E_YELLOW, OUTPUT);
  pinMode(E_GREEN, OUTPUT);
  pinMode(W_RED, OUTPUT);
  pinMode(W_YELLOW, OUTPUT);
```

```
  pinMode(W_GREEN, OUTPUT);

  // Initialize Button Pins with internal pull-up
  pinMode(BTN_MENU, INPUT_PULLUP);
  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);

  // Start with all red
  allRed();

  lcd.clear();
}

void loop() {
  // Check buttons
  checkButtons();

  if (inConfigMode) {
    handleConfigMenu();
  } else {
    // Normal traffic light operation
    runTrafficSequence();
    displayStatus();
  }
}

void runTrafficSequence() {
  DateTime now = rtc.now();

  // Check if current time is in peak hours
  int currentHour = now.hour();
  isPeakHour = (currentHour >= peakStartHour && currentHour <
 ↪  peakEndHour);
  currentGreenTime = isPeakHour ? peakGreenTime : normalGreenTime;

  unsigned long currentMillis = millis();

  // Update phase based on timer
  if (currentMillis - previousMillis >= 1000) { // Every second
    previousMillis = currentMillis;
    phaseTimer++;

    // Phase transitions
    switch (currentPhase) {
      case 0: // North/South Green
        if (phaseTimer >= currentGreenTime) {
          phaseTimer = 0;
          currentPhase = 1;
```

```
        }
        break;

      case 1: // North/South Yellow
        if (phaseTimer >= yellowTime) {
          phaseTimer = 0;
          currentPhase = 2;
        }
        break;

      case 2: // All Red (transition)
        if (phaseTimer >= allRedTime) {
          phaseTimer = 0;
          currentPhase = 3;
        }
        break;

      case 3: // East/West Green
        if (phaseTimer >= currentGreenTime) {
          phaseTimer = 0;
          currentPhase = 4;
        }
        break;

      case 4: // East/West Yellow
        if (phaseTimer >= yellowTime) {
          phaseTimer = 0;
          currentPhase = 5;
        }
        break;

      case 5: // All Red (transition)
        if (phaseTimer >= allRedTime) {
          phaseTimer = 0;
          currentPhase = 0;
        }
        break;
    }

    // Update lights based on current phase
    updateLights();
  }
}

void updateLights() {
  switch (currentPhase) {
    case 0: // North/South Green, East/West Red
      digitalWrite(N_GREEN, HIGH);
```

```
    digitalWrite(N_YELLOW, LOW);
    digitalWrite(N_RED, LOW);
    digitalWrite(S_GREEN, HIGH);
    digitalWrite(S_YELLOW, LOW);
    digitalWrite(S_RED, LOW);
    digitalWrite(E_RED, HIGH);
    digitalWrite(E_YELLOW, LOW);
    digitalWrite(E_GREEN, LOW);
    digitalWrite(W_RED, HIGH);
    digitalWrite(W_YELLOW, LOW);
    digitalWrite(W_GREEN, LOW);
    break;

  case 1: // North/South Yellow, East/West Red
    digitalWrite(N_GREEN, LOW);
    digitalWrite(N_YELLOW, HIGH);
    digitalWrite(S_GREEN, LOW);
    digitalWrite(S_YELLOW, HIGH);
    break;

  case 2: // All Red
    allRed();
    break;

  case 3: // East/West Green, North/South Red
    digitalWrite(N_RED, HIGH);
    digitalWrite(N_YELLOW, LOW);
    digitalWrite(N_GREEN, LOW);
    digitalWrite(S_RED, HIGH);
    digitalWrite(S_YELLOW, LOW);
    digitalWrite(S_GREEN, LOW);
    digitalWrite(E_GREEN, HIGH);
    digitalWrite(E_YELLOW, LOW);
    digitalWrite(E_RED, LOW);
    digitalWrite(W_GREEN, HIGH);
    digitalWrite(W_YELLOW, LOW);
    digitalWrite(W_RED, LOW);
    break;

  case 4: // East/West Yellow, North/South Red
    digitalWrite(E_GREEN, LOW);
    digitalWrite(E_YELLOW, HIGH);
    digitalWrite(W_GREEN, LOW);
    digitalWrite(W_YELLOW, HIGH);
    break;

  case 5: // All Red
    allRed();
```

```
      break;
  }
}

void allRed() {
  digitalWrite(N_RED, HIGH);
  digitalWrite(N_YELLOW, LOW);
  digitalWrite(N_GREEN, LOW);
  digitalWrite(S_RED, HIGH);
  digitalWrite(S_YELLOW, LOW);
  digitalWrite(S_GREEN, LOW);
  digitalWrite(E_RED, HIGH);
  digitalWrite(E_YELLOW, LOW);
  digitalWrite(E_GREEN, LOW);
  digitalWrite(W_RED, HIGH);
  digitalWrite(W_YELLOW, LOW);
  digitalWrite(W_GREEN, LOW);
}

void displayStatus() {
  DateTime now = rtc.now();

  lcd.setCursor(0, 0);
  lcd.print(now.hour() < 10 ? "0" : "");
  lcd.print(now.hour());
  lcd.print(":");
  lcd.print(now.minute() < 10 ? "0" : "");
  lcd.print(now.minute());
  lcd.print(" ");
  lcd.print(isPeakHour ? "PEAK" : "NORM");
  lcd.print("   ");

  lcd.setCursor(0, 1);
  switch (currentPhase) {
    case 0:
      lcd.print("N/S:GRN E/W:RED ");
      break;
    case 1:
      lcd.print("N/S:YEL E/W:RED ");
      break;
    case 2:
      lcd.print("ALL RED        ");
      break;
    case 3:
      lcd.print("E/W:GRN N/S:RED ");
      break;
    case 4:
      lcd.print("E/W:YEL N/S:RED ");
```

```
      break;
    case 5:
      lcd.print("ALL RED        ");
      break;
  }
}

void checkButtons() {
  unsigned long currentTime = millis();

  if (currentTime - lastButtonPress < buttonDebounce) {
    return;
  }

  if (digitalRead(BTN_MENU) == LOW) {
    lastButtonPress = currentTime;
    if (!inConfigMode) {
      inConfigMode = true;
      currentMenu = MENU_MAIN;
      lcd.clear();
    } else {
      // Cycle through menu options
      currentMenu = (MenuState)((currentMenu + 1) % 6);

      if (currentMenu == MENU_MAIN) {
        inConfigMode = false;
        lcd.clear();
      }
    }
  }

  if (inConfigMode) {
    if (digitalRead(BTN_UP) == LOW) {
      lastButtonPress = currentTime;
      adjustValue(1);
    }

    if (digitalRead(BTN_DOWN) == LOW) {
      lastButtonPress = currentTime;
      adjustValue(-1);
    }
  }
}

void adjustValue(int direction) {
  switch (currentMenu) {
    case MENU_PEAK_GREEN:
      peakGreenTime = constrain(peakGreenTime + direction, 5, 120);
```

```
      break;
    case MENU_NORMAL_GREEN:
      normalGreenTime = constrain(normalGreenTime + direction, 5, 120);
      break;
    case MENU_YELLOW:
      yellowTime = constrain(yellowTime + direction, 1, 10);
      break;
    case MENU_PEAK_START:
      peakStartHour = constrain(peakStartHour + direction, 0, 23);
      break;
    case MENU_PEAK_END:
      peakEndHour = constrain(peakEndHour + direction, 0, 23);
      break;
    default:
      break;
  }
}

void handleConfigMenu() {
  lcd.setCursor(0, 0);

  switch (currentMenu) {
    case MENU_MAIN:
      lcd.print("Config Menu     ");
      lcd.setCursor(0, 1);
      lcd.print("Press Menu Btn  ");
      break;

    case MENU_PEAK_GREEN:
      lcd.print("Peak Green Time ");
      lcd.setCursor(0, 1);
      lcd.print(peakGreenTime);
      lcd.print(" sec    ");
      break;

    case MENU_NORMAL_GREEN:
      lcd.print("Normal Grn Time ");
      lcd.setCursor(0, 1);
      lcd.print(normalGreenTime);
      lcd.print(" sec    ");
      break;

    case MENU_YELLOW:
      lcd.print("Yellow Time     ");
      lcd.setCursor(0, 1);
      lcd.print(yellowTime);
      lcd.print(" sec    ");
      break;
```

```
    case MENU_PEAK_START:
      lcd.print("Peak Start Hour ");
      lcd.setCursor(0, 1);
      lcd.print(peakStartHour);
      lcd.print(":00    ");
      break;

    case MENU_PEAK_END:
      lcd.print("Peak End Hour   ");
      lcd.setCursor(0, 1);
      lcd.print(peakEndHour);
      lcd.print(":00    ");
      break;
  }
}
```

JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": 342.6, "left":
     ↪  540.4, "attrs": {} },
    { "type": "wokwi-breadboard-mini", "id": "bb2", "top": 94.6, "left":
     ↪  -88.8, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 29.4, "left":
     ↪  508.2, "attrs": {} },
    { "type": "wokwi-led", "id": "led1", "top": -138, "left": -34.6,
     ↪  "attrs": { "color": "red" } },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": -90,
      "left": -34.6,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led3",
      "top": -32.4,
      "left": -34.6,
      "attrs": { "color": "green", "flip": "" }
    },
    {
```

```
      "type": "wokwi-led",
      "id": "led4",
      "top": 284.4,
      "left": -34.6,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-led",
      "id": "led5",
      "top": 351.6,
      "left": -34.6,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led6",
      "top": 418.8,
      "left": -34.6,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-led",
      "id": "led7",
      "top": 130.8,
      "left": -351.4,
      "attrs": { "color": "red" }
    },
    {
      "type": "wokwi-led",
      "id": "led8",
      "top": 130.8,
      "left": -293.8,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led9",
      "top": 130.8,
      "left": -236.2,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-led",
      "id": "led10",
      "top": 121.2,
      "left": 282.2,
      "attrs": { "color": "red" }
    },
```

```
    {
      "type": "wokwi-led",
      "id": "led11",
      "top": 121.2,
      "left": 215,
      "attrs": { "color": "yellow" }
    },
    {
      "type": "wokwi-led",
      "id": "led12",
      "top": 121.2,
      "left": 157.4,
      "attrs": { "color": "green", "flip": "" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -101.65,
      "left": 9.6,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -53.65,
      "left": 9.6,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": 4.25,
      "left": 8.2,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r4",
      "top": 196.8,
      "left": 153.05,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r5",
      "top": 196.8,
      "left": 210.65,
```

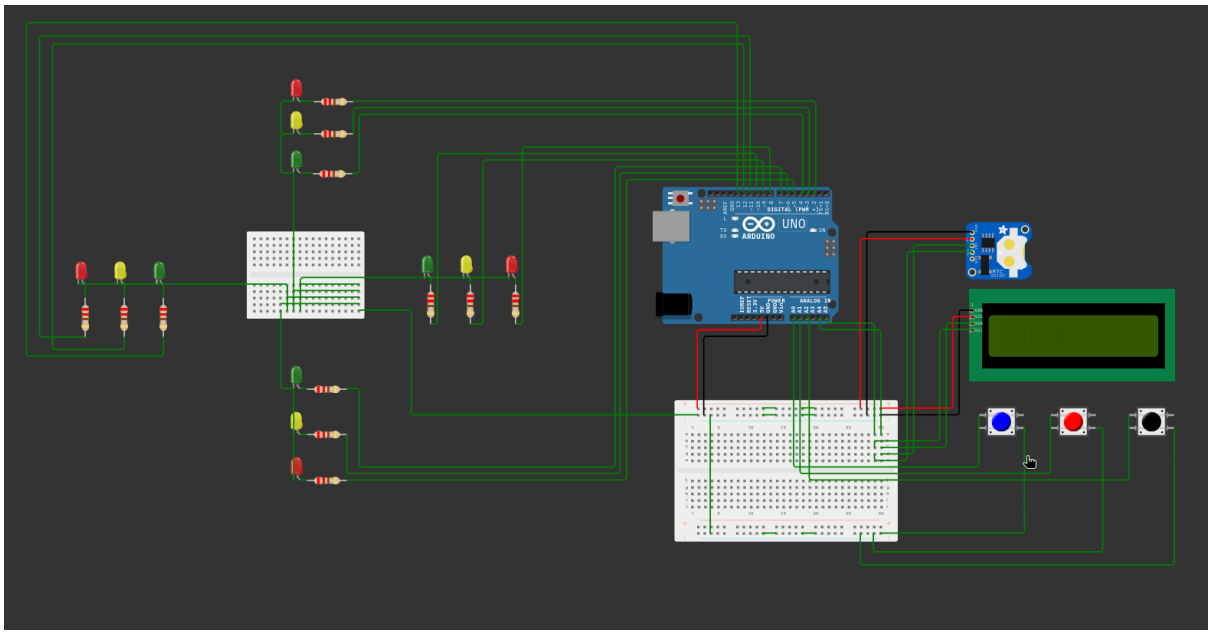
```
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r6",
      "top": 196.8,
      "left": 277.85,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r7",
      "top": 320.75,
      "left": 0,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r8",
      "top": 387.95,
      "left": 0,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r9",
      "top": 455.15,
      "left": 0,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r10",
      "top": 216,
      "left": -355.75,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r11",
      "top": 216,
      "left": -298.15,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
```

```
    {
      "type": "wokwi-resistor",
      "id": "r12",
      "top": 216,
      "left": -240.55,
      "rotate": 90,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": 179.2,
      "left": 975.2,
      "attrs": { "pins": "i2c" }
    },
    { "type": "wokwi-ds1307", "id": "rtc1", "top": 81, "left": 969.7,
     ↪  "attrs": {} },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 351.8,
      "left": 988.8,
      "attrs": { "color": "blue", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 351.8,
      "left": 1094.4,
      "attrs": { "color": "red", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn3",
      "top": 351.8,
      "left": 1209.6,
      "attrs": { "color": "black", "xray": "1" }
    }
  ],
  "connections": [
    [ "led7:A", "r10:1", "green", [ "v0" ] ],
    [ "led8:A", "r11:1", "green", [ "v0" ] ],
    [ "led9:A", "r12:1", "green", [ "v0" ] ],
    [ "led4:A", "r7:1", "green", [ "v0" ] ],
    [ "led5:A", "r8:1", "green", [ "v0" ] ],
    [ "led6:A", "r9:1", "green", [ "v0" ] ],
    [ "led12:A", "r4:1", "green", [ "v0" ] ],
    [ "led11:A", "r5:1", "green", [ "v0" ] ],
```

```
    [ "led10:A", "r6:1", "green", [ "v0" ] ],
    [ "led3:A", "r3:1", "green", [ "v0" ] ],
    [ "led2:A", "r2:1", "green", [ "v0" ] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "led1:C", "led2:C", "green", [ "h-18.8", "v48" ] ],
    [ "led3:C", "led2:C", "green", [ "h-18.8", "v-57.6" ] ],
    [ "led3:C", "bb2:7b.j", "green", [ "v0" ] ],
    [ "led11:C", "led12:C", "green", [ "v0" ] ],
    [ "led10:C", "led11:C", "green", [ "v0" ] ],
    [ "led12:C", "bb2:8b.j", "green", [ "h-124.4", "v0", "h-57.6" ] ],
    [ "led7:C", "led8:C", "green", [ "v0" ] ],
    [ "led8:C", "led9:C", "green", [ "v0" ] ],
    [ "led9:C", "bb2:6b.j", "green", [ "h192.4", "v38.4" ] ],
    [ "led6:C", "led5:C", "green", [ "v0" ] ],
    [ "led4:C", "led5:C", "green", [ "v0" ] ],
    [ "led4:C", "bb2:5b.j", "green", [ "h-18.8", "v-115.2" ] ],
    [ "r1:2", "uno:2", "green", [ "h680.4", "v134.4" ] ],
    [ "r2:2", "uno:3", "green", [ "v-38.4", "h670.8", "v124.8" ] ],
    [ "r3:2", "uno:4", "green", [ "h9.8", "v-86.7", "h652.8", "v115.2" ]
    ↪ ],
    [ "r9:2", "uno:5", "green", [ "h411.6", "v-441.6", "h245.8" ] ],
    [ "r8:2", "uno:6", "green", [ "v57.6", "h402", "v-441.6", "h245.9" ]
    ↪ ],
    [ "r7:2", "uno:7", "green", [ "h18", "v115.2", "h376.8", "v-441.6",
    ↪ "h243.6" ] ],
    [ "r6:2", "uno:8", "green", [ "h9.6", "v-260.4", "h364.4" ] ],
    [ "r5:2", "uno:9", "green", [ "h19.2", "v-241.2", "h412" ] ],
    [ "r4:2", "uno:10", "green", [ "h9.6", "v-250.8", "h469.7" ] ],
    [ "r10:2", "uno:11", "green", [ "h-67.2", "v-442.8", "h1045.8" ] ],
    [ "r11:2", "uno:12", "green", [ "v18", "h-105.6", "v-448.8",
    ↪ "h1017.1" ] ],
    [ "r12:2", "uno:13", "green", [ "v27.6", "h-201.6", "v-489.6",
    ↪ "h1046" ] ],
    [ "uno:5V", "bb1:tp.1", "red", [ "v19.1", "h-93.8" ] ],
    [ "bb2:17b.j", "bb1:tn.1", "green", [ "h76.8", "v153.7" ] ],
    [ "uno:GND.2", "bb1:tn.2", "black", [ "v28.7", "h-93.7" ] ],
    [ "bb1:tn.10", "bb1:tn.11", "green", [ "v0" ] ],
    [ "bb1:tp.10", "bb1:tp.11", "green", [ "v0" ] ],
    [ "bb1:tp.15", "bb1:tp.16", "green", [ "v0" ] ],
    [ "bb1:tn.15", "bb1:tn.16", "green", [ "v0" ] ],
    [ "lcd1:GND", "bb1:tn.25", "black", [ "h-19.2", "v153.7" ] ],
    [ "lcd1:VCC", "bb1:tp.25", "red", [ "h-28.8", "v134.5", "h-107.2" ]
    ↪ ],
    [ "lcd1:SDA", "bb1:30t.c", "green", [ "h-38.4", "v182.6" ] ],
    [ "lcd1:SCL", "bb1:29t.c", "green", [ "h-48", "v163.5", "h-96" ] ],
    [ "rtc1:GND", "bb1:tn.23", "black", [ "h0" ] ],
    [ "rtc1:5V", "bb1:tp.22", "red", [ "h0" ] ],
    [ "rtc1:SDA", "bb1:30t.d", "green", [ "h-86.4", "v306.9" ] ],
```

```
    [ "rtc1:SCL", "bb1:29t.d", "green", [ "h-96", "v307", "h-48" ] ],
    [ "bb1:30t.a", "uno:A4", "green", [ "v-153.6", "h-90.6" ] ],
    [ "bb1:29t.a", "uno:A5", "green", [ "v-163.2", "h-71.5" ] ],
    [ "btn1:2.r", "bb1:bn.25", "green", [ "h0.2", "v155.1" ] ],
    [ "btn2:2.r", "bb1:bn.24", "green", [ "h9.8", "v182.6", "h-337.6" ]
     ↪  ],
    [ "btn3:2.r", "bb1:bn.22", "green", [ "h0.2", "v201.8", "h-462.4" ]
     ↪  ],
    [ "bb1:tn.3", "bb1:bn.3", "green", [ "v0" ] ],
    [ "bb1:bn.10", "bb1:bn.11", "green", [ "v0" ] ],
    [ "bb1:bn.15", "bb1:bn.16", "green", [ "v0" ] ],
    [ "btn1:1.l", "uno:A0", "green", [ "v76.8", "h-272.6" ] ],
    [ "btn2:1.l", "uno:A1", "green", [ "v86.4", "h-368.7" ] ],
    [ "btn3:1.l", "uno:A2", "green", [ "v96", "h-470.4", "v-239.9" ] ],
    [ "bb2:17b.f", "bb2:8b.f", "green", [ "v0" ] ],
    [ "bb2:17b.g", "bb2:7b.g", "green", [ "v0" ] ],
    [ "bb2:17b.h", "bb2:6b.h", "green", [ "v0" ] ],
    [ "bb2:17b.i", "bb2:5b.i", "green", [ "v0" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 3	ODOMETER
04/01/2026	

AIM

To simulate the odometer(distance calculator)

APPARATUS REQUIRED

- Arduino Uno
- Breadboard
- Stepper Motor (NEMA 17)
- A4988 Stepper Motor Driver
- I2C LCD Display (16x2)
- Pushbuttons (x4) (Start/Stop, Increase Speed, Decrease Speed, Reset)
- Jumper wires

PRE-CONDITIONS

- The stepper motor, A4988 driver module, pushbuttons, and I2C LCD are wired securely to the Arduino.
- The A4988 driver is connected with proper microstepping pins configured (MS1, MS2, MS3) and appropriate power.
- The `Wire.h` and `LiquidCrystal_I2C.h` libraries are installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The stepper motor acts as a wheel simulator, turning at the specified target speed.
- The LCD accurately displays the simulated speed (m/s) and total distance traveled (m).
- The pushbuttons effectively start/stop the motor, adjust the simulated target speed, and reset the distance counter.
- System handles speed adjustments gracefully without sudden jumps.

CODE

```
float targetSpeed = 0.5;          // m/s (adjustable)
float maxSpeed = 5000.0;             // m/s (increased limit)
float minSpeed = 0.1;             // m/s
float speedStep = 0.2;            // m/s increment#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// === PIN DEFINITIONS ===
#define STEP_PIN 8
```

```
#define DIR_PIN 9
#define ENABLE_PIN 10
#define RESET_PIN 11

#define BTN1_PIN 2   // Start/Stop
#define BTN2_PIN 3   // Increase Speed
#define BTN3_PIN 4   // Decrease Speed
#define BTN4_PIN 5   // Reset Distance

// Track button states
volatile bool btn2Pressed = false;
volatile bool btn3Pressed = false;
volatile bool btn4Pressed = false;

// === STEPPER MOTOR PARAMETERS ===
#define STEPS_PER_REV 200        // 1.8° per step
#define MICROSTEPS 16            // A4988 microstepping (MS1, MS2, MS3
 ↪  high)
#define WHEEL_DIAMETER 0.1       // 10cm diameter in meters
#define WHEEL_CIRCUMFERENCE (PI * WHEEL_DIAMETER)
#define STEPS_PER_METER (STEPS_PER_REV * MICROSTEPS) /
 ↪  WHEEL_CIRCUMFERENCE

// === LCD SETUP ===
LiquidCrystal_I2C lcd(0x27, 16, 2); // Address 0x27, 16x2 display

// === VARIABLES ===
volatile long totalSteps = 0;
volatile unsigned long lastStepTime = 0;
unsigned long lastDisplayUpdate = 0;
unsigned long lastSpeedCalcTime = 0;

float currentSpeed = 0.0;        // m/s
float totalDistance = 0.0;       // meters


bool motorRunning = false;
bool speedChanged = false;

// Button debounce variables
unsigned long lastBtnTime[4] = {0, 0, 0, 0};
#define DEBOUNCE_DELAY 200

// === INTERRUPT HANDLERS ===
void btn1ISR() {
  if (millis() - lastBtnTime[0] < DEBOUNCE_DELAY) return;
  lastBtnTime[0] = millis();
  motorRunning = !motorRunning;
```

```
  if (motorRunning) {
    digitalWrite(ENABLE_PIN, LOW);  // Enable motor
  } else {
    digitalWrite(ENABLE_PIN, HIGH); // Disable motor
  }
}

void btn2ISR() {
  btn2Pressed = true;
}

void btn3ISR() {
  btn3Pressed = true;
}

void btn4ISR() {
  btn4Pressed = true;
}

// === SETUP ===
void setup() {
  Serial.begin(9600);

  // Initialize pins
  pinMode(STEP_PIN, OUTPUT);
  pinMode(DIR_PIN, OUTPUT);
  pinMode(ENABLE_PIN, OUTPUT);
  pinMode(RESET_PIN, OUTPUT);

  pinMode(BTN1_PIN, INPUT_PULLUP);
  pinMode(BTN2_PIN, INPUT_PULLUP);
  pinMode(BTN3_PIN, INPUT_PULLUP);
  pinMode(BTN4_PIN, INPUT_PULLUP);

  // Initialize motor control
  digitalWrite(RESET_PIN, HIGH);   // Release reset
  digitalWrite(ENABLE_PIN, HIGH);  // Disable motor initially
  digitalWrite(DIR_PIN, HIGH);     // Set direction (CCW)

  // Initialize LCD
  lcd.init();
  lcd.backlight();
  lcd.print("Odometer Ready!");
  delay(2000);
  lcd.clear();

  // Attach interrupts
```

```
  attachInterrupt(digitalPinToInterrupt(BTN1_PIN), btn1ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(BTN2_PIN), btn2ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(BTN3_PIN), btn3ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(BTN4_PIN), btn4ISR, FALLING);

  lastDisplayUpdate = millis();
  lastSpeedCalcTime = millis();
}

// === MAIN LOOP ===
void loop() {
  // Handle button presses with debouncing
  handleButtonInputs();

  // Step the motor if running
  if (motorRunning) {
    stepMotor();
  }

  // Calculate speed (steps per second to m/s)
  unsigned long currentTime = millis();
  if (currentTime - lastSpeedCalcTime >= 500) {
    updateSpeed(currentTime - lastSpeedCalcTime);
    lastSpeedCalcTime = currentTime;
  }

  // Update LCD display
  if (currentTime - lastDisplayUpdate >= 200) {
    updateLCD();
    lastDisplayUpdate = currentTime;
  }
}

// === BUTTON HANDLER ===
void handleButtonInputs() {
  static unsigned long lastBtn2Time = 0;
  static unsigned long lastBtn3Time = 0;
  static unsigned long lastBtn4Time = 0;
  unsigned long currentTime = millis();

  // Handle button 2 (Increase Speed)
  if (btn2Pressed) {
    if (currentTime - lastBtn2Time >= DEBOUNCE_DELAY) {
      targetSpeed += speedStep;
      if (targetSpeed > maxSpeed) targetSpeed = maxSpeed;
      speedChanged = true;
      lastBtn2Time = currentTime;
    }
```

```
    btn2Pressed = false;
  }

  // Handle button 3 (Decrease Speed)
  if (btn3Pressed) {
    if (currentTime - lastBtn3Time >= DEBOUNCE_DELAY) {
      targetSpeed -= speedStep;
      if (targetSpeed < minSpeed) targetSpeed = minSpeed;
      speedChanged = true;
      lastBtn3Time = currentTime;
    }
    btn3Pressed = false;
  }

  // Handle button 4 (Reset Distance)
  if (btn4Pressed) {
    if (currentTime - lastBtn4Time >= DEBOUNCE_DELAY) {
      totalSteps = 0;
      totalDistance = 0.0;
      currentSpeed = 0.0;
      lastBtn4Time = currentTime;
    }
    btn4Pressed = false;
  }

  // Print speed if changed
  if (speedChanged) {
    Serial.print("Target Speed: ");
    Serial.print(targetSpeed, 2);
    Serial.println(" m/s");
    speedChanged = false;
  }
}

// === STEPPER MOTOR CONTROL ===
void stepMotor() {
  // Calculate delay based on target speed
  // Speed (m/s) = steps/second / steps_per_meter
  // steps/second = speed * steps_per_meter
  float stepsPerSecond = targetSpeed * STEPS_PER_METER;
  float delayMicros = 1000000.0 / stepsPerSecond;

  unsigned long currentMicros = micros();

  // Only step if enough time has passed
  if (currentMicros - lastStepTime >= delayMicros) {
    // Pulse the STEP pin
    digitalWrite(STEP_PIN, HIGH);
```

```
    delayMicroseconds(5);
    digitalWrite(STEP_PIN, LOW);

    lastStepTime = currentMicros;
    totalSteps++;

    // Update distance (in meters)
    totalDistance = (float)totalSteps / STEPS_PER_METER;
  }
}

// === SPEED CALCULATION ===
void updateSpeed(unsigned long deltaTime) {
  // Current speed based on recent step rate
  // This is smoothed over the last update interval
  float deltaSeconds = deltaTime / 1000.0;

  // If moving, report target speed; otherwise 0
  if (motorRunning && targetSpeed > 0) {
    currentSpeed = targetSpeed;
  } else {
    currentSpeed = 0.0;
  }
}

// === LCD UPDATE ===
void updateLCD() {
  // Line 1: Speed
  lcd.setCursor(0, 0);
  lcd.print("Speed:");
  lcd.setCursor(7, 0);
  lcd.print(currentSpeed, 2);
  lcd.print(" m/s ");

  // Line 2: Distance
  lcd.setCursor(0, 1);
  lcd.print("Dist:");
  lcd.setCursor(6, 1);
  lcd.print(totalDistance, 2);
  lcd.print("m   ");

  // Debug output to Serial
  Serial.print("Speed: ");
  Serial.print(currentSpeed, 2);
  Serial.print(" m/s | Distance: ");
  Serial.print(totalDistance, 2);
  Serial.print(" m | Running: ");
  Serial.println(motorRunning ? "YES" : "NO");
```

```
}

// === UTILITY FUNCTIONS ===
void displayStatus(const char* msg) {
  lcd.clear();
  lcd.print(msg);
  delay(1000);
}
```

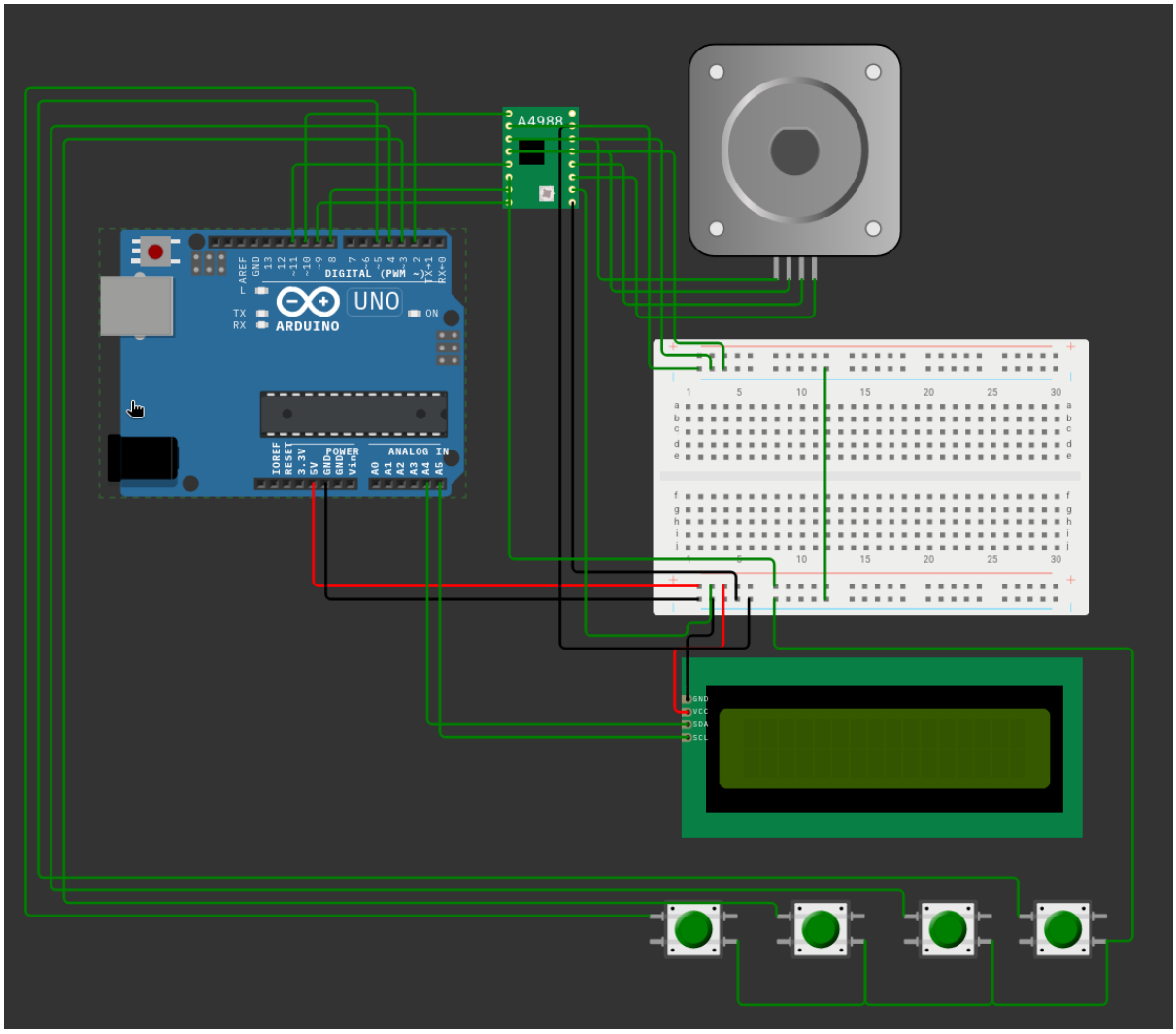
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": 141, "left":
     ↪  175.6, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 58.2, "left":
     ↪  -240.6, "attrs": {} },
    {
      "type": "wokwi-stepper-motor",
      "id": "stepper1",
      "top": -82.79,
      "left": 202.03,
      "attrs": { "size": "17" }
    },
    { "type": "wokwi-a4988", "id": "drv1", "top": -33.6, "left": 62.4,
     ↪  "attrs": {} },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": 380.8,
      "left": 197.6,
      "attrs": { "pins": "i2c" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 563,
      "left": 172.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 563,
```

```
      "left": 268.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn3",
      "top": 563,
      "left": 364.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn4",
      "top": 563,
      "left": 451.2,
      "attrs": { "color": "green", "xray": "1" }
    }
  ],
  "connections": [
    [ "uno:5V", "bb1:bp.1", "red", [ "v0" ] ],
    [ "uno:GND.2", "bb1:bn.1", "black", [ "v0" ] ],
    [ "lcd1:GND", "bb1:bn.2", "black", [ "v-48", "h19.2", "v-46.7" ] ],
    [ "lcd1:VCC", "bb1:bp.3", "red", [ "h-9.6", "v-47.9", "h36.8" ] ],
    [ "lcd1:SCL", "uno:A5", "green", [ "h0" ] ],
    [ "lcd1:SDA", "uno:A4", "green", [ "h0" ] ],
    [ "drv1:ENABLE", "uno:10", "green", [ "h-153.6", "v96" ] ],
    [ "drv1:RESET", "uno:11", "green", [ "h-163.2", "v57.6" ] ],
    [ "drv1:STEP", "uno:8", "green", [ "h0" ] ],
    [ "drv1:DIR", "uno:9", "green", [ "h0" ] ],
    [ "drv1:GND.1", "bb1:bn.4", "black", [ "v278.48", "h123.35" ] ],
    [ "drv1:GND.2", "bb1:bn.5", "black", [ "h-9.45", "v393.68", "h142.4"
    ↪  ] ],
    [ "drv1:VDD", "bb1:bp.2", "green", [ "h9.75", "v336.08", "h76.8",
    ↪  "v-9.6", "h17.6" ] ],
    [ "stepper1:A-", "drv1:2B", "green", [ "h-134.4", "v-105.68" ] ],
    [ "stepper1:A+", "drv1:2A", "green", [ "v9.6", "h-134.4", "v-105.68"
    ↪  ] ],
    [ "stepper1:B+", "drv1:1A", "green", [ "v19.2", "h-134.4", "v-105.68"
    ↪  ] ],
    [ "stepper1:B-", "drv1:1B", "green", [ "v28.8", "h-134.41",
    ↪  "v-105.68" ] ],
    [ "bb1:bn.10", "bb1:tn.10", "green", [ "v0" ] ],
    [ "drv1:MS1", "bb1:tn.1", "green", [ "h105.6", "v182.5" ] ],
    [ "drv1:MS2", "bb1:tn.2", "green", [ "h115.2", "v163.2", "h36.8" ] ],
    [ "drv1:MS3", "bb1:tn.3", "green", [ "h124.8", "v144", "h36.8" ] ],
    [ "drv1:SLEEP", "bb1:bp.6", "green", [ "v288", "h200" ] ],
    [ "btn1:2.r", "btn2:2.r", "green", [ "v48.2", "h96" ] ],
    [ "btn2:2.r", "btn3:2.r", "green", [ "v48.2", "h96" ] ],
```

```
    [ "btn3:2.r", "btn4:2.r", "green", [ "v48.2", "h86.4" ] ],
    [ "btn4:2.r", "bb1:bn.6", "green", [ "h19.4", "v-220.6", "h-270.4" ]
     ↪  ],
    [ "btn1:1.l", "uno:2", "green", [ "h-470.4", "v-624", "h293.5" ] ],
    [ "uno:3", "btn2:1.l", "green", [ "v-76.8", "h-255.2", "v576",
     ↪  "h537.6" ] ],
    [ "uno:4", "btn3:1.l", "green", [ "v-86.4", "h-255.3", "v576",
     ↪  "h643.2" ] ],
    [ "uno:5", "btn4:1.l", "green", [ "v-105.6", "h-255.4", "v585.6",
     ↪  "h739.2" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 4	MUSIC VISUALIZER
30/01/2026	

AIM

To simulate the Music Visualizer

APPARATUS REQUIRED

- Arduino Uno
- MAX7219 8x8 LED Matrix Module
- 10-Segment LED Bar Graph
- Buzzer
- Resistors (x10)
- Pushbuttons (x2) (Next, Previous)
- Jumper wires

PRE-CONDITIONS

- The LED matrix is correctly connected via SPI (DIN, CS, CLK).
- The 10-Segment LED Bar Graph is properly wired with current-limiting resistors to the Arduino digital and analog pins.
- The buzzer and pushbuttons are appropriately connected.
- The `LedControl.h` library is installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino Uno board.

POST-CONDITIONS

- The buzzer plays melodies based on predefined musical notes and duration.
- The LED Bar Graph lights up proportionally to the frequency of the current note being played.
- The 8x8 LED Matrix visualizes the audio frequencies using dynamic bar heights and simulated harmonics.
- The pushbuttons allow the user to smoothly switch between different songs in the playlist.

CODE

```
#include <LedControl.h>

LedControl matrix = LedControl(11, 13, 10, 1);

const int ledPins[] = {2, 3, 4, 5, 6, 7, A0, A1, A2, A3};
const int ledCount = 10;
const int buzzerPin = 12;
```

```
const int btnNext = 8;
const int btnPrev = 9;

#define NOTE_B0  31
#define NOTE_C1  33
#define NOTE_CS1 35
#define NOTE_D1  37
#define NOTE_DS1 39
#define NOTE_E1  41
#define NOTE_F1  44
#define NOTE_FS1 46
#define NOTE_G1  49
#define NOTE_GS1 52
#define NOTE_A1  55
#define NOTE_AS1 58
#define NOTE_B1  62
#define NOTE_C2  65
#define NOTE_CS2 69
#define NOTE_D2  73
#define NOTE_DS2 78
#define NOTE_E2  82
#define NOTE_F2  87
#define NOTE_FS2 93
#define NOTE_G2  98
#define NOTE_GS2 104
#define NOTE_A2  110
#define NOTE_AS2 117
#define NOTE_B2  123
#define NOTE_C3  131
#define NOTE_CS3 139
#define NOTE_D3  147
#define NOTE_DS3 156
#define NOTE_E3  165
#define NOTE_F3  175
#define NOTE_FS3 185
#define NOTE_G3  196
#define NOTE_GS3 208
#define NOTE_A3  220
#define NOTE_AS3 233
#define NOTE_B3  247
#define NOTE_C4  262
#define NOTE_CS4 277
#define NOTE_D4  294
#define NOTE_DS4 311
#define NOTE_E4  330
#define NOTE_F4  349
#define NOTE_FS4 370
#define NOTE_G4  392
```

```
#define NOTE_GS4 415
#define NOTE_A4  440
#define NOTE_AS4 466
#define NOTE_B4  494
#define NOTE_C5  523
#define NOTE_CS5 554
#define NOTE_D5  587
#define NOTE_DS5 622
#define NOTE_E5  659
#define NOTE_F5  698
#define NOTE_FS5 740
#define NOTE_G5  784
#define NOTE_GS5 831
#define NOTE_A5  880
#define NOTE_AS5 932
#define NOTE_B5  988
#define NOTE_C6  1047
#define NOTE_CS6 1109
#define NOTE_D6  1175
#define NOTE_DS6 1245
#define NOTE_E6  1319
#define NOTE_F6  1397
#define NOTE_FS6 1480
#define NOTE_G6  1568
#define NOTE_GS6 1661
#define NOTE_A6  1760
#define NOTE_AS6 1865
#define NOTE_B6  1976
#define NOTE_C7  2093
#define NOTE_CS7 2217
#define NOTE_D7  2349
#define NOTE_DS7 2489
#define NOTE_E7  2637
#define NOTE_F7  2794
#define NOTE_FS7 2960
#define NOTE_G7  3136
#define NOTE_GS7 3322
#define NOTE_A7  3520
#define NOTE_AS7 3729
#define NOTE_B7  3951
#define NOTE_C8  4186
#define NOTE_CS8 4435
#define NOTE_D8  4699
#define NOTE_DS8 4978
#define REST 0

struct Note {
  int frequency;
```

```
  int duration;
};

struct Song {
  const char* name;
  const char* album;
  Note* notes;
  int length;
};

Note marioTheme[] = {
  {NOTE_E5, 150}, {NOTE_E5, 150}, {REST, 150}, {NOTE_E5, 150},
  {REST, 150}, {NOTE_C5, 150}, {NOTE_E5, 150}, {REST, 150},
  {NOTE_G5, 150}, {REST, 450}, {NOTE_G4, 150}, {REST, 450},

  {NOTE_C5, 150}, {REST, 300}, {NOTE_G4, 150}, {REST, 300},
  {NOTE_E4, 150}, {REST, 300}, {NOTE_A4, 150}, {REST, 150},
  {NOTE_B4, 150}, {REST, 150}, {NOTE_AS4, 150}, {NOTE_A4, 150},
  {REST, 150},

  {NOTE_G4, 120}, {NOTE_E5, 120}, {NOTE_G5, 120}, {NOTE_A5, 150},
  {REST, 150}, {NOTE_F5, 150}, {NOTE_G5, 150}, {REST, 150},
  {NOTE_E5, 150}, {REST, 150}, {NOTE_C5, 150}, {NOTE_D5, 150},
  {NOTE_B4, 150}, {REST, 300}
};

Note marioUnderworld[] = {
  {NOTE_C4, 150}, {NOTE_C5, 150}, {NOTE_A3, 150}, {NOTE_A4, 150},
  {NOTE_AS3, 150}, {NOTE_AS4, 150}, {REST, 300},

  {NOTE_C4, 150}, {NOTE_C5, 150}, {NOTE_A3, 150}, {NOTE_A4, 150},
  {NOTE_AS3, 150}, {NOTE_AS4, 150}, {REST, 300},

  {NOTE_F3, 150}, {NOTE_F4, 150}, {NOTE_D3, 150}, {NOTE_D4, 150},
  {NOTE_DS3, 150}, {NOTE_DS4, 150}, {REST, 300},

  {NOTE_F3, 150}, {NOTE_F4, 150}, {NOTE_D3, 150}, {NOTE_D4, 150},
  {NOTE_DS3, 150}, {NOTE_DS4, 150}, {REST, 300},

  {NOTE_DS4, 225}, {NOTE_CS4, 225}, {NOTE_D4, 225},
  {NOTE_CS4, 150}, {NOTE_DS4, 150}, {NOTE_DS4, 150}, {NOTE_GS3, 150},
  {NOTE_G3, 150}, {NOTE_CS4, 150}
};

Note marioPowerUp[] = {
  {NOTE_G4, 100}, {NOTE_B4, 100}, {NOTE_D5, 100}, {NOTE_G5, 100},
  {NOTE_B5, 100}, {NOTE_G5, 100}, {NOTE_B5, 100}, {NOTE_D6, 100},
  {NOTE_G6, 100}, {NOTE_B6, 100}, {NOTE_G6, 100}, {NOTE_B6, 100},
```

```
  {NOTE_D7, 100}, {REST, 200}
};

Note marioCoin[] = {
  {NOTE_B5, 100}, {NOTE_E6, 400}, {REST, 200}
};

Note marioGameOver[] = {
  {NOTE_C5, 150}, {NOTE_G4, 150}, {NOTE_E4, 150},
  {NOTE_A4, 225}, {NOTE_B4, 225}, {NOTE_A4, 225},
  {NOTE_GS4, 225}, {NOTE_AS4, 225}, {NOTE_GS4, 225},
  {NOTE_G4, 150}, {NOTE_D4, 150}, {NOTE_E4, 150}, {REST, 300}
};

Note marioJump[] = {
  {NOTE_C6, 75}, {NOTE_E6, 75}, {NOTE_G6, 75}, {NOTE_C7, 75}
};

Note zeldaTheme[] = {
  {NOTE_AS4, 200}, {NOTE_F4, 75}, {NOTE_F4, 75}, {NOTE_AS4, 150},
  {NOTE_GS4, 200}, {NOTE_FS4, 200}, {NOTE_GS4, 200},
  {NOTE_AS4, 300}, {NOTE_FS4, 150}, {NOTE_GS4, 150},
  {NOTE_AS4, 200}, {NOTE_FS4, 75}, {NOTE_FS4, 75},

  {NOTE_AS4, 150}, {NOTE_A4, 200}, {NOTE_F4, 200},
  {NOTE_AS4, 400}, {REST, 200},

  {NOTE_AS4, 200}, {NOTE_F4, 75}, {NOTE_F4, 75}, {NOTE_AS4, 150},
  {NOTE_GS4, 200}, {NOTE_FS4, 200}, {NOTE_GS4, 200},
  {NOTE_AS4, 300}, {NOTE_FS4, 150}, {NOTE_GS4, 150},
  {NOTE_AS4, 200}, {NOTE_FS4, 75}, {NOTE_FS4, 75},

  {NOTE_AS4, 150}, {NOTE_CS5, 200}, {NOTE_DS5, 200},
  {NOTE_E5, 200}, {NOTE_FS5, 150}, {NOTE_E5, 150},
  {NOTE_DS5, 200}, {NOTE_CS5, 150}, {NOTE_DS5, 150},
  {NOTE_E5, 200}, {NOTE_CS5, 150}, {NOTE_DS5, 150},

  {NOTE_E5, 150}, {NOTE_CS5, 150}, {NOTE_C5, 150}, {NOTE_CS5, 150},
  {NOTE_D5, 200}, {NOTE_E5, 200}, {NOTE_FS5, 150},
  {NOTE_E5, 150}, {NOTE_DS5, 200}, {NOTE_CS5, 150},
  {NOTE_DS5, 150}, {NOTE_E5, 200}, {NOTE_CS5, 150},

  {NOTE_DS5, 150}, {NOTE_E5, 150}, {NOTE_CS5, 150},
  {NOTE_C5, 150}, {NOTE_CS5, 150}, {NOTE_D5, 200},
  {NOTE_E5, 200}, {NOTE_FS5, 150}, {NOTE_E5, 150},
  {NOTE_DS5, 200}, {NOTE_CS5, 150}, {NOTE_DS5, 150},

  {NOTE_E5, 200}, {NOTE_CS5, 150}, {NOTE_C5, 200},
```

```
  {NOTE_CS5, 600}, {REST, 200}
};

Song playlist[] = {
  {"Main Theme", "Super Mario Bros", marioTheme, 38},
  {"Underworld", "Super Mario Bros", marioUnderworld, 43},
  {"Power Up", "Super Mario Bros", marioPowerUp, 14},
  {"Coin Sound", "Super Mario Bros", marioCoin, 3},
  {"Game Over", "Super Mario Bros", marioGameOver, 13},
  {"Jump Sound", "Super Mario Bros", marioJump, 4},
  {"Main Theme", "Legend of Zelda", zeldaTheme, 70}
};

int currentSong = 0;
int totalSongs = 7;
int noteIndex = 0;
unsigned long lastNote = 0;
bool isPlaying = false;
int barHeights[8] = {0, 0, 0, 0, 0, 0, 0, 0};
int targetHeights[8] = {0, 0, 0, 0, 0, 0, 0, 0};

void setup() {
  for (int i = 0; i < ledCount; i++) {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], LOW);
  }
  pinMode(btnNext, INPUT_PULLUP);
  pinMode(btnPrev, INPUT_PULLUP);
  pinMode(buzzerPin, OUTPUT);

  matrix.shutdown(0, false);
  matrix.setIntensity(0, 8);
  matrix.clearDisplay(0);

  isPlaying = true;
}

void loop() {
  if (digitalRead(btnNext) == LOW) {
    stopSong();
    currentSong = (currentSong + 1) % totalSongs;
    resetSong();
    delay(300);
  }

  if (digitalRead(btnPrev) == LOW) {
    stopSong();
    currentSong = (currentSong - 1 + totalSongs) % totalSongs;
```

```
    resetSong();
    delay(300);
  }

  if (isPlaying) {
    Song* song = &playlist[currentSong];

    if (millis() - lastNote >= song->notes[noteIndex].duration) {
      playCurrentNote();
      noteIndex++;

      if (noteIndex >= song->length) {
        noteIndex = 0;
      }
      lastNote = millis();
    }
  }

  updateMatrixBars();
}

void playCurrentNote() {
  Song* song = &playlist[currentSong];
  int freq = song->notes[noteIndex].frequency;

  if (freq == REST) {
    noTone(buzzerPin);
    updateLEDBar(0);
    for (int i = 0; i < 8; i++) {
      targetHeights[i] = 0;
    }
  } else {
    tone(buzzerPin, freq);

    int level = map(freq, 31, 4978, 1, ledCount);
    level = constrain(level, 1, ledCount);
    updateLEDBar(level);

    updateMatrixTargets(freq, level);
  }
}

void updateMatrixTargets(int freq, int level) {
  int baseHeight = map(level, 0, ledCount, 0, 8);

  for (int i = 0; i < 8; i++) {
    int variance = random(-2, 3);
    targetHeights[i] = constrain(baseHeight + variance, 0, 8);
```

```
  }

  int harmonic1 = map(freq % 500, 0, 500, 0, 8);
  int harmonic2 = map((freq * 2) % 800, 0, 800, 0, 8);

  targetHeights[1] = constrain(targetHeights[1] + harmonic1 / 2, 0, 8);
  targetHeights[4] = constrain(targetHeights[4] + harmonic2 / 2, 0, 8);
  targetHeights[6] = constrain(targetHeights[6] + harmonic1 / 3, 0, 8);
}

void updateMatrixBars() {
  matrix.clearDisplay(0);

  for (int col = 0; col < 8; col++) {
    if (barHeights[col] < targetHeights[col]) {
      barHeights[col]++;
    } else if (barHeights[col] > targetHeights[col]) {
      barHeights[col]--;
    }

    for (int row = 0; row < 8; row++) {
      if (row < barHeights[col]) {
        matrix.setLed(0, 7 - row, col, true);
      }
    }
  }
}

void updateLEDBar(int level) {
  for (int i = 0; i < ledCount; i++) {
    if (i < level) {
      digitalWrite(ledPins[i], HIGH);
    } else {
      digitalWrite(ledPins[i], LOW);
    }
  }
}

void stopSong() {
  noTone(buzzerPin);
  for (int i = 0; i < ledCount; i++) {
    digitalWrite(ledPins[i], LOW);
  }
  for (int i = 0; i < 8; i++) {
    targetHeights[i] = 0;
    barHeights[i] = 0;
  }
  matrix.clearDisplay(0);
```

```
}

void resetSong() {
  noteIndex = 0;
  matrix.clearDisplay(0);
  isPlaying = true;
  for (int i = 0; i < 8; i++) {
    targetHeights[i] = 0;
    barHeights[i] = 0;
  }
}
```

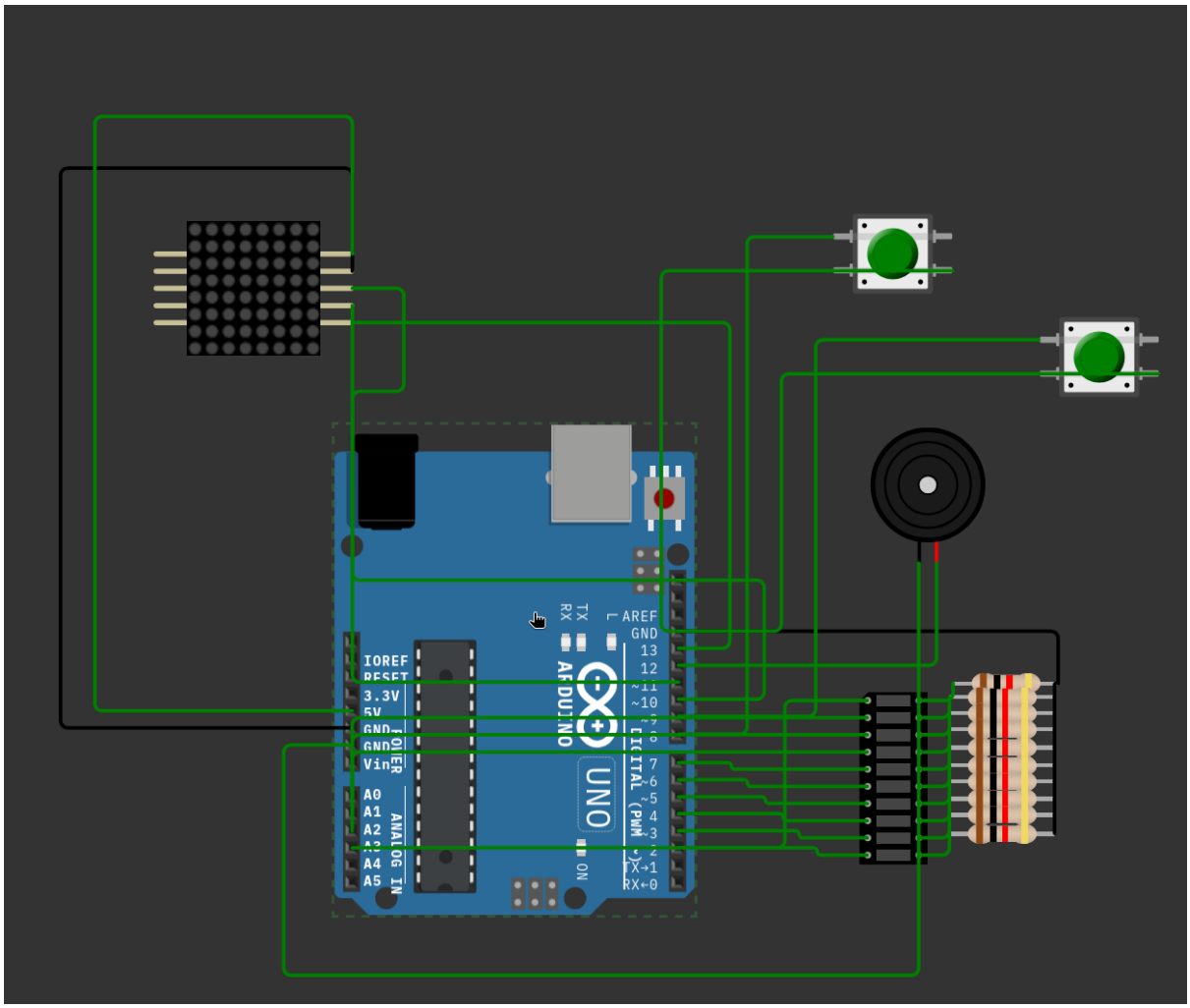
JSON

```
{
  "version": 1,
  "author": "Uri Shaked",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 208.2,
      "left": -190.6,
      "rotate": 90,
      "attrs": {}
    },
    {
      "type": "wokwi-led-bar-graph",
      "id": "bargraph1",
      "top": 321.6,
      "left": 139.2,
      "attrs": { "color": "BCYR" }
    },
    { "type": "wokwi-resistor", "id": "r1", "top": 311.15, "left": 192,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r2", "top": 319, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r3", "top": 328, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r4", "top": 337, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r5", "top": 347, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r6", "top": 357, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r7", "top": 366, "left": 190,
     ↪  "attrs": {} },
```

```
    { "type": "wokwi-resistor", "id": "r8", "top": 376, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r9", "top": 385, "left": 190,
     ↪  "attrs": {} },
    { "type": "wokwi-resistor", "id": "r10", "top": 395, "left": 190,
     ↪  "attrs": {} },
    {
      "type": "wokwi-max7219-matrix",
      "id": "matrix1",
      "top": 58.2,
      "left": -255.39,
      "attrs": { "chain": "1" }
    },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": 165.6,
      "left": 145.8,
      "attrs": { "volume": "0.1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 54.2,
      "left": 124.8,
      "attrs": { "color": "green", "xray": "1" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 111.8,
      "left": 240,
      "attrs": { "color": "green", "xray": "1" }
    }
  ],
  "connections": [
    [ "bargraph1:A10", "uno:2", "green", [ "h-28", "v31" ] ],
    [ "bargraph1:A9", "uno:3", "green", [ "h-38", "v31" ] ],
    [ "bargraph1:A8", "uno:4", "green", [ "h-47", "v31" ] ],
    [ "bargraph1:A7", "uno:5", "green", [ "h-57", "v31" ] ],
    [ "bargraph1:A6", "uno:6", "green", [ "h-67", "v31" ] ],
    [ "bargraph1:A5", "uno:7", "green", [ "h-76", "v31" ] ],
    [ "r1:1", "bargraph1:C1", "green", [ "v0" ] ],
    [ "r2:1", "bargraph1:C2", "green", [ "v0" ] ],
    [ "r3:1", "bargraph1:C3", "green", [ "v0" ] ],
    [ "r4:1", "bargraph1:C4", "green", [ "v0" ] ],
    [ "r5:1", "bargraph1:C5", "green", [ "v0" ] ],
    [ "r6:1", "bargraph1:C6", "green", [ "v0" ] ],
```

```
    [ "r7:1", "bargraph1:C7", "green", [ "v0" ] ],
    [ "r8:1", "bargraph1:C8", "green", [ "v0" ] ],
    [ "r9:1", "bargraph1:C9", "green", [ "v0" ] ],
    [ "r10:1", "bargraph1:C10", "green", [ "v0" ] ],
    [ "r10:2", "r9:2", "black", [ "v0" ] ],
    [ "r8:2", "r9:2", "black", [ "v0" ] ],
    [ "r8:2", "r7:2", "black", [ "v0" ] ],
    [ "r7:2", "r6:2", "black", [ "v0" ] ],
    [ "r6:2", "r5:2", "black", [ "v0" ] ],
    [ "r5:2", "r4:2", "black", [ "v0" ] ],
    [ "r4:2", "r3:2", "black", [ "v0" ] ],
    [ "r3:2", "r2:2", "black", [ "v0" ] ],
    [ "r2:2", "r1:2", "black", [ "v0" ] ],
    [ "uno:GND.1", "r1:2", "black", [ "h0" ] ],
    [ "matrix1:GND", "uno:GND.2", "black", [ "v-57.6", "h-393.6",
     ↪  "v312.9" ] ],
    [ "matrix1:V+", "uno:5V", "green", [ "v-76.8", "h-364.8", "v332.2" ]
     ↪  ],
    [ "matrix1:DIN", "uno:11", "green", [ "h28.8", "v57.6", "h-28.8",
     ↪  "v162.6" ] ],
    [ "matrix1:CS", "uno:10", "green", [ "v153.6", "h230.4", "v66.5" ] ],
    [ "matrix1:CLK", "uno:13", "green", [ "h211.2", "v182" ] ],
    [ "bz1:2", "uno:12", "green", [ "v0" ] ],
    [ "bz1:1", "uno:GND.3", "green", [ "v230.4", "h-355.2", "v-128.8" ]
     ↪  ],
    [ "bargraph1:A4", "uno:A0", "green", [ "h0" ] ],
    [ "bargraph1:A3", "uno:A1", "green", [ "h0" ] ],
    [ "bargraph1:A2", "uno:A2", "green", [ "h0" ] ],
    [ "bargraph1:A1", "uno:A3", "green", [ "h-46.08", "v90.86" ] ],
    [ "btn1:1.l", "uno:8", "green", [ "h-48", "v278" ] ],
    [ "btn2:1.l", "uno:9", "green", [ "h-124.8", "v210.4" ] ],
    [ "btn1:2.r", "uno:GND.1", "green", [ "h-163", "v77" ] ],
    [ "btn2:2.r", "uno:GND.1", "green", [ "h-211", "v143.9" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 5	MOTION DETECTION AND PHOTO RESISTOR
30/01/2026	

AIM

To simulate the motion detection and usage of photo resistor

APPARATUS REQUIRED

- Arduino Uno
- Breadboard
- PIR Motion Sensor
- Photoresistor (LDR) Sensor Module
- LEDs (x3) (White, Cyan, Orange)
- 220-ohm Resistors (x3)
- Jumper wires

PRE-CONDITIONS

- The PIR sensor, LDR module, and LEDs (with resistors) are correctly wired to the designated Arduino pins.
- The LDR module's analog (AO) and digital (DO) outputs are properly connected.
- The code is successfully compiled and uploaded to the Arduino Uno board.
- Serial Monitor is opened at 9600 baud rate to view the debug messages.

POST-CONDITIONS

- The White LED activates when the PIR sensor detects motion.
- The Cyan LED's brightness dynamically adjusts (PWM) based on the ambient light detected by the LDR's analog pin.
- The Orange LED activates exclusively when the LDR's digital threshold drops indicating a dark environment.
- The Serial Monitor displays continuous real-time status of motion detection, calculated lux levels, and system diagnostics.

CODE

```
const int PIR_PIN = 2;
const int PHOTO_DIGITAL_PIN = 3;
const int PHOTO_ANALOG_PIN = A0;

const int MOTION_LED = 8;   // White LED
const int ANALOG_LED = 9;   // Blue LED
const int DIGITAL_LED = 10; // Orange LED
```

```
const int LIGHT_THRESHOLD = 512;

void setup()
{
  Serial.begin(9600);

  pinMode(PIR_PIN, INPUT);
  pinMode(PHOTO_DIGITAL_PIN, INPUT);
  pinMode(MOTION_LED, OUTPUT);
  pinMode(ANALOG_LED, OUTPUT);
  pinMode(DIGITAL_LED, OUTPUT);

  Serial.println("========================================");
  Serial.println("Motion & Photoresistor Sensor Demo");
  Serial.println("========================================");
  Serial.println("White LED (Pin 8)  - Motion Detection");
  Serial.println("Blue LED (Pin 9)   - Analog Light Level");
  Serial.println("Orange LED (Pin 10) - Digital Light Status");
  Serial.println("========================================");
  delay(2000);
  Serial.println("System Ready!\n");
}

void loop()
{
  Serial.println("========================================");

  // PIR Motion Sensor
  int motionDetected = digitalRead(PIR_PIN);

  if (motionDetected == HIGH)
  {
    digitalWrite(MOTION_LED, HIGH);
    Serial.println(">>> MOTION DETECTED! <<<");
  }
  else
  {
    digitalWrite(MOTION_LED, LOW);
    Serial.println("Motion: None");
  }

  // Photoresistor Analog Output
  int analogValue = analogRead(PHOTO_ANALOG_PIN);

  // Exponential curve for visible brightness changes
  float normalizedValue = (1023.0 - analogValue) / 1023.0;
  float gamma = 2.5;
  float correctedValue = pow(normalizedValue, gamma);
```

```
int ledBrightness = (int)(correctedValue * 255.0);
ledBrightness = constrain(ledBrightness, 0, 255);

analogWrite(ANALOG_LED, ledBrightness);

// Convert to lux
const float GAMMA_LUX = 0.7;
const float RL10 = 50;
float voltage = analogValue / 1024.0 * 5.0;
float resistance = 2000.0 * voltage / (1.0 - voltage / 5.0);
float lux = pow(RL10 * 1e3 * pow(10, GAMMA_LUX) / resistance, (1.0 /
 ↪  GAMMA_LUX));

Serial.println("\n--- ANALOG OUTPUT (Blue LED) ---");
Serial.print("Raw Value: ");
Serial.print(analogValue);
Serial.print(" | Voltage: ");
Serial.print(voltage, 2);
Serial.print("V | Light: ");

if (isfinite(lux))
{
  Serial.print((int)lux);
  Serial.println(" lux");
}
else
{
  Serial.println("Very Bright");
}

Serial.print("Brightness: ");
Serial.print(ledBrightness);
Serial.print("/255 (");
Serial.print((ledBrightness * 100) / 255);
Serial.print("%) ");

if (ledBrightness < 25)
{
  Serial.println("- OFF/VERY DIM");
}
else if (ledBrightness < 80)
{
  Serial.println("- DIM");
}
else if (ledBrightness < 150)
{
  Serial.println("- MEDIUM");
}
```

```
else if (ledBrightness < 220)
{
  Serial.println("- BRIGHT");
}
else
{
  Serial.println("- VERY BRIGHT");
}

Serial.print("Bar: [");
int bars = map(ledBrightness, 0, 255, 0, 20);
for (int i = 0; i < bars; i++)
  Serial.print(" ");
for (int i = bars; i < 20; i++)
  Serial.print(" ");
Serial.println("]");

// Photoresistor Digital Output
int digitalValue = digitalRead(PHOTO_DIGITAL_PIN);

if (digitalValue == LOW)
{
  digitalWrite(DIGITAL_LED, HIGH);
  Serial.println("\n--- DIGITAL OUTPUT (Orange LED) ---");
  Serial.println("Status: BRIGHT | Orange LED: ON");
}
else
{
  digitalWrite(DIGITAL_LED, LOW);
  Serial.println("\n--- DIGITAL OUTPUT (Orange LED) ---");
  Serial.println("Status: DARK | Orange LED: OFF");
}

// Custom Threshold
Serial.println("\n--- CUSTOM THRESHOLD ---");
if (analogValue > LIGHT_THRESHOLD)
{
  Serial.print("Value (");
  Serial.print(analogValue);
  Serial.print(") > (");
  Serial.print(LIGHT_THRESHOLD);
  Serial.println(") = DARK");
}
else
{
  Serial.print("Value (");
  Serial.print(analogValue);
  Serial.print(")   (");
```

```
    Serial.print(LIGHT_THRESHOLD);
    Serial.println(") = BRIGHT");
  }

  Serial.println("========================================\n");
  delay(1000);
}
```

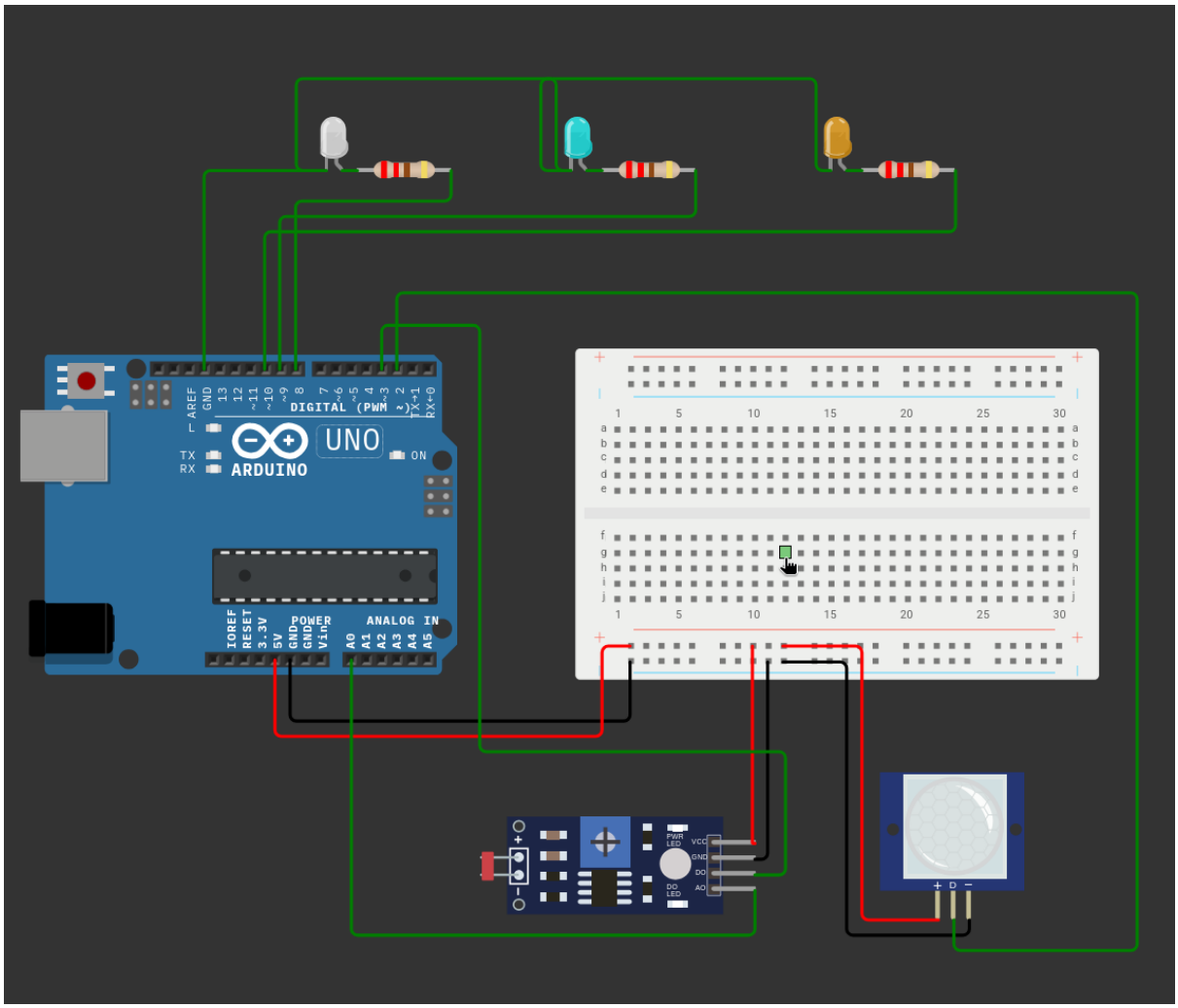
JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-half", "id": "bb1", "top": -60.6, "left":
     ↪  233.2, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top": -57, "left":
     ↪  -115.8, "attrs": {} },
    { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": 205.6,
     ↪  "left": 424.62, "attrs": {} },
    {
      "type": "wokwi-photoresistor-sensor",
      "id": "ldr1",
      "top": 233.6,
      "left": 173.6,
      "attrs": {}
    },
    {
      "type": "wokwi-led",
      "id": "led1",
      "top": -214.8,
      "left": 61.4,
      "attrs": { "color": "white" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -178.45,
      "left": 96,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-led",
      "id": "led2",
      "top": -214.8,
      "left": 215,
```

```
      "attrs": { "color": "cyan" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -178.45,
      "left": 249.6,
      "attrs": { "value": "220" }
    },
    {
      "type": "wokwi-led",
      "id": "led3",
      "top": -214.8,
      "left": 378.2,
      "attrs": { "color": "orange" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -178.45,
      "left": 412.8,
      "attrs": { "value": "220" }
    }
  ],
  "connections": [
    [ "uno:GND.2", "bb1:bn.1", "black", [ "v38.3", "h213.5" ] ],
    [ "uno:5V", "bb1:bp.1", "red", [ "v47.9", "h205.4", "v-56.7" ] ],
    [ "pir1:VCC", "bb1:bp.10", "red", [ "h-48", "v-171.9" ] ],
    [ "pir1:GND", "bb1:bn.10", "black", [ "v9.6", "h-77.06", "v-171.5" ]
     ↪ ],
    [ "pir1:OUT", "uno:2", "green", [ "v19.2", "h115.06", "v-412.8",
     ↪ "h-464.9" ] ],
    [ "ldr1:VCC", "bb1:bp.8", "red", [ "h0" ] ],
    [ "ldr1:GND", "bb1:bn.9", "black", [ "h0" ] ],
    [ "ldr1:DO", "uno:3", "green", [ "h19.2", "v-77.4", "h-192",
     ↪ "v-267.6", "h-61.6" ] ],
    [ "ldr1:AO", "uno:A0", "green", [ "v28.1", "h-253.4" ] ],
    [ "led1:A", "r1:1", "green", [ "v0" ] ],
    [ "led2:A", "r2:1", "green", [ "v0" ] ],
    [ "led3:A", "r3:1", "green", [ "v0" ] ],
    [ "led1:C", "led2:C", "green", [ "v0", "h-18.8", "v-57.6", "h163.2",
     ↪ "v57.6" ] ],
    [ "led2:C", "led3:C", "green", [ "v0", "h-18.8", "v-57.6", "h172.8",
     ↪ "v57.6" ] ],
    [ "r1:2", "uno:8", "green", [ "v19.2", "h-1.2" ] ],
    [ "r2:2", "uno:9", "green", [ "v28.8", "h-1.2" ] ],
    [ "r3:2", "uno:10", "green", [ "v38.4", "h-433.2" ] ],
    [ "led1:C", "uno:GND.1", "green", [ "v0", "h-76.4" ] ]
```

```
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION



EX.NO: 6	GAS LEAKAGE DETECTION
06/02/2026	

AIM

To simulate the gas leakage detection

APPARATUS REQUIRED

- Arduino Uno
- Gas Sensor (MQ Series)
- RGB LED
- Buzzer
- Relay Module
- I2C OLED Display (128x64)
- 200-ohm Resistors (x3)
- Jumper wires

PRE-CONDITIONS

- The Gas Sensor, RGB LED, Buzzer, Relay module, and I2C OLED display are correctly wired to the Arduino.
- Libraries `Wire.h`, `Adafruit_GFX.h`, and `Adafruit_SSD1306.h` are installed in the Arduino IDE.
- The code is successfully compiled and uploaded to the Arduino board.
- The sensor has had appropriate time to preheat if dealing with a real hardware MQ sensor.

POST-CONDITIONS

- The OLED screen displays the real-time gas concentration values and system status (SAFE, WARNING, CRITICAL).
- In a SAFE state, the RGB LED glows green, the buzzer is off, and the relay (exhaust) is deactivated.
- In a WARNING state, the RGB LED glows yellow and the buzzer beeps intermittently.
- In a CRITICAL state, the RGB LED glows red, the buzzer sounds a continuous alarm, and the relay activates the exhaust fan.

CODE

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

```
// OLED Configuration
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET     -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

const int gasSensorPin = A0;
const int buzzerPin = 8;
const int redPin = 11;
const int greenPin = 10;
const int bluePin = 9;
const int relayPin = 6;    // NEW: Relay Control Pin

const int SAFE_LIMIT = 350;
const int WARNING_LIMIT = 600;

int gasValue = 0;

void setup() {
  Serial.begin(9600);

  pinMode(gasSensorPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
  pinMode(bluePin, OUTPUT);
  pinMode(relayPin, OUTPUT); // NEW: Set Relay as output

  // Start with Relay OFF (Active LOW usually means LOW = ON, but let's
  ↪  assume HIGH = OFF)
  digitalWrite(relayPin, HIGH);

  // Initialize OLED
  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
  display.setCursor(0,0);
  display.println("Sys Initialize...");
  display.println("Exhaust: OFF");
  display.display();
  delay(2000);

  setColor(0, 255, 0);
```

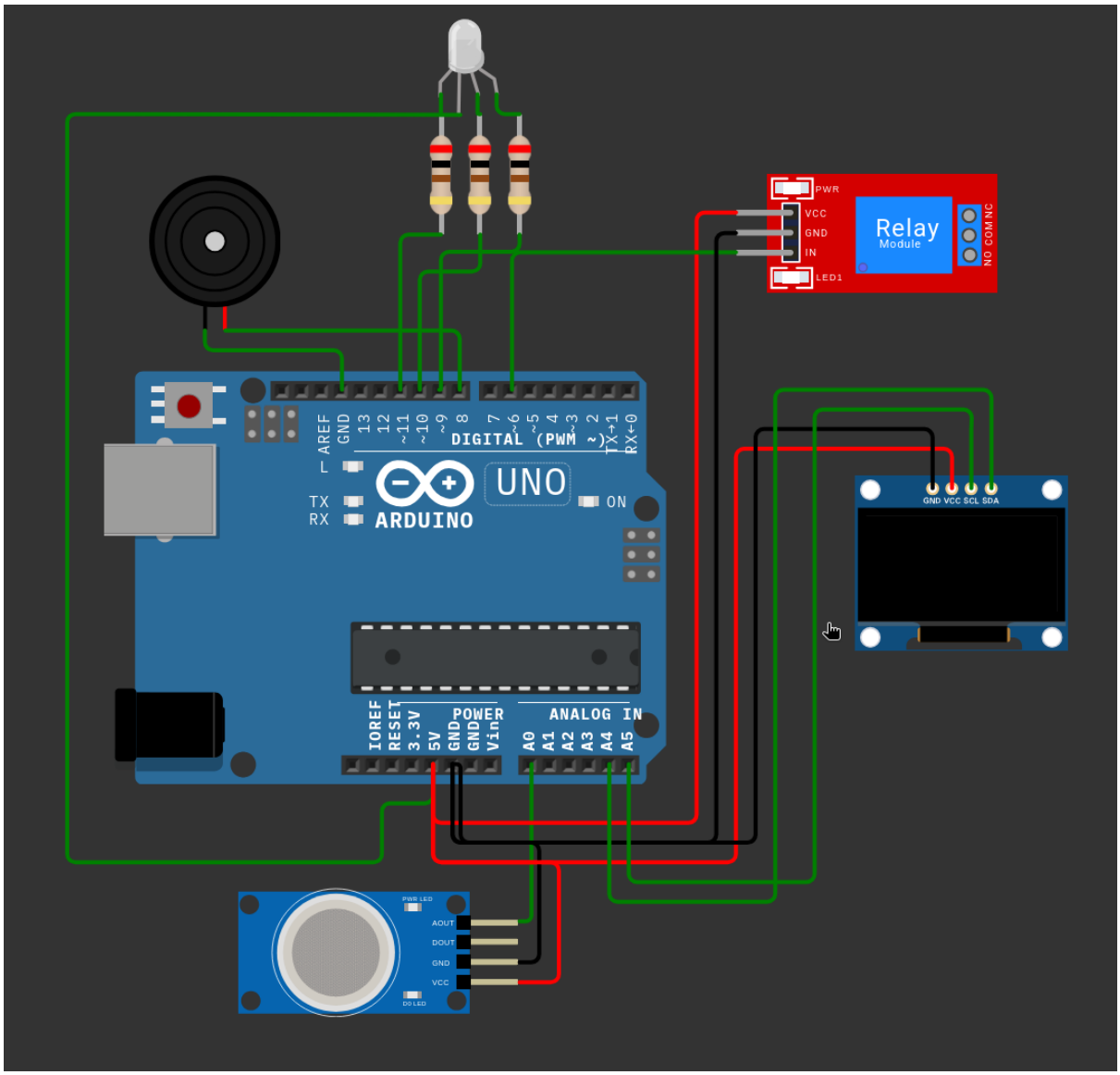
```
  digitalWrite(buzzerPin, LOW);
}

void loop() {
  gasValue = analogRead(gasSensorPin);

  display.clearDisplay();
  display.setCursor(0,0);

  // Header
  display.setTextSize(1);
  display.println("Gas Safety Unit");
  display.println("----------------");
  display.print("Gas Val: ");
  display.print(gasValue);
  display.println("   ");

  if (gasValue >= WARNING_LIMIT) {
    // --- DANGER: ACTIVATE SAFETY SYSTEMS ---

    display.setTextColor(BLACK, WHITE);
    display.println(" ALERT: CRITICAL");
    display.setTextColor(WHITE);

    // SAFETY ACTION: Turn Relay ON (Active LOW logic used here)
    digitalWrite(relayPin, LOW);
    display.println(">> EXHAUST: ON <<"); // Fan is spinning

    setColor(255, 0, 0);

    // Alarm
    tone(buzzerPin, 1000);
    delay(200);
    noTone(buzzerPin);
    delay(200);

  } else if (gasValue >= SAFE_LIMIT) {
    // --- WARNING ---

    display.println(" ALERT: WARNING");

    // Keep Exhaust OFF to save power (or ON if you prefer constant
     ↪  ventilation)
    digitalWrite(relayPin, HIGH);
    display.println("Exhaust: STANDBY");

    setColor(255, 255, 0);
```

```
    tone(buzzerPin, 800);
    delay(150);
    noTone(buzzerPin);
    delay(850);

  } else {
    // --- SAFE ---

    display.println(" STATUS: SAFE");

    // SAFETY ACTION: Turn Relay OFF
    digitalWrite(relayPin, HIGH);
    display.println("Exhaust: OFF");

    setColor(0, 255, 0);

    digitalWrite(buzzerPin, LOW);
    delay(500);
  }

  display.display();
}

void setColor(int redValue, int greenValue, int blueValue) {
  analogWrite(redPin, 255 - redValue);
  analogWrite(greenPin, 255 - greenValue);
  analogWrite(bluePin, 255 - blueValue);
}
```

JSON

```
{
  "version": 1,
  "author": "Shree Kottes",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-arduino-uno", "id": "uno", "top": 77.4, "left":
     ↪  -10.2, "attrs": {} },
    { "type": "wokwi-gas-sensor", "id": "gas1", "top": 329.1, "left": 55,
     ↪  "attrs": {} },
    { "type": "wokwi-rgb-led", "id": "rgb1", "top": -101.6, "left":
     ↪  145.1, "attrs": {} },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": -26.4,
      "left": 11.4,
```

```
      "attrs": { "volume": "0.1" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r1",
      "top": -24,
      "left": 162.65,
      "rotate": 90,
      "attrs": { "value": "200" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -24,
      "left": 124.25,
      "rotate": 90,
      "attrs": { "value": "200" }
    },
    {
      "type": "wokwi-resistor",
      "id": "r3",
      "top": -24,
      "left": 143.45,
      "rotate": 90,
      "attrs": { "value": "200" }
    },
    { "type": "wokwi-relay-module", "id": "relay1", "top": -19, "left":
     ↪  297.6, "attrs": {} },
    {
      "type": "board-ssd1306",
      "id": "oled1",
      "top": 127.94,
      "left": 355.43,
      "attrs": { "i2cAddress": "0x3c" }
    }
  ],
  "connections": [
    [ "gas1:AOUT", "uno:A0", "green", [ "h0" ] ],
    [ "gas1:VCC", "uno:5V", "red", [ "h19.2", "v-58.5", "h-57.6" ] ],
    [ "gas1:GND", "uno:GND.2", "black", [ "h9.6", "v-58.4", "h-38.4" ] ],
    [ "bz1:1", "uno:GND.1", "green", [ "v9.6", "h57.6" ] ],
    [ "bz1:2", "uno:8", "green", [ "v0", "h114.8" ] ],
    [ "rgb1:R", "r2:1", "green", [ "v0" ] ],
    [ "rgb1:G", "r3:1", "green", [ "v0" ] ],
    [ "rgb1:B", "r1:1", "green", [ "v0" ] ],
    [ "r2:2", "uno:11", "green", [ "h0" ] ],
    [ "r3:2", "uno:10", "green", [ "v18", "h-29.5" ] ],
    [ "r1:2", "uno:9", "green", [ "h0", "v8.4", "h-38.4", "v67.2" ] ],
```

```
    [
      "rgb1:COM",
      "uno:5V",
      "green",
      [ "v-0.4", "h-191.9", "v364.8", "h153.6", "v-28.8", "h19.2" ]
    ],
    [ "relay1:VCC", "uno:5V", "red", [ "h-19.2", "v297.6", "h-124.8" ] ],
    [ "relay1:IN", "uno:6", "green", [ "h0" ] ],
    [ "relay1:GND", "uno:GND.2", "black", [ "h-9.6", "v297.2", "h-124.8"
     ↪  ] ],
    [ "oled1:VCC", "uno:5V", "red", [ "v-19.2", "h-105.45", "v201.6",
     ↪  "h-153.6" ] ],
    [ "oled1:GND", "uno:GND.2", "black", [ "v-28.8", "h-86.4", "v201.6",
     ↪  "h-144", "v-9.6" ] ],
    [
      "oled1:SCL",
      "uno:A5",
      "green",
      [ "v-28.8", "h0.3", "v-9.6", "h-76.8", "v230.4", "h-86.4" ]
    ],
    [ "oled1:SDA", "uno:A4", "green", [ "v-48", "h-105.53", "v249.6",
     ↪  "h-86.4" ] ]
  ],
  "dependencies": {}
}
```

PIN CONFIGURATION